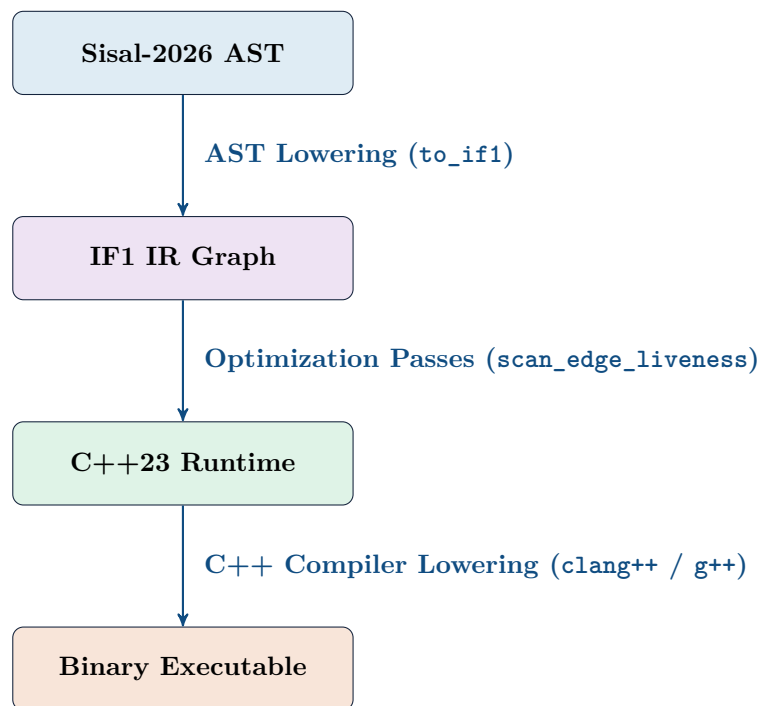


Language Specification & Reference Manual



Features: Dense Dope Vectors (`array_dv`) • Rank Polymorphism • $O(1)$ Zero-Copy Views • Einstein Summation (EINSUM) • 58 APL Bulk Operators • Stackless Coroutine Streams • Monad Order Sequencing

Sisal-2026 Compiler Team

LLNL / UEA Heritage · Modern Optimizing Compiler Infrastructure

September 10, 2026

Contents

Preface	iv
1 Introduction to Sisal-2026	1
1.1 Overview & Dataflow Parallelism	1
1.2 Core Principles & Architectural Heritage	1
1.3 Sisal-2026 Language Innovations	2
1.4 Structure of this Specification	3
2 Practical Guide: Writing Common Programs in Sisal-2026	4
2.1 The Sisal-2026 Programming Model	4
2.2 Program 1: "Hello, World!" & I/O	4
2.3 Program 2: Vector Arithmetic & Rank Polymorphism	5
2.4 Program 3: Matrix Multiplication (EINSUM vs. Parallel Loops)	5
2.5 Program 4: Iterative State Algorithms (<code>for initial</code>)	6
2.6 Program 5: Lazy Coroutine Streams (<code>stream[T]</code>)	7
2.7 Program 6: Image Processing with Bulk Stencils	7
2.8 Program 7: Modern Transformer Layer (RMSNorm, GQA, & SwiGLU FFN)	8
2.9 Program 8: Hand-Optimized Fused Transformer Layer	9
2.9.1 Further Fusion Opportunities: Joint GEMM, Inline Exponentials, and Complete 3-Stage FFN Fusion	11
2.10 Program 9: Fully Inlined & Fused Transformer Block	12
2.11 Compiling & Running Sisal-2026 Programs	14
2.11.1 Step 1: Translate Sisal Source to C++23	15
2.11.2 Step 2: Create a C++ Driver (<code>main.cpp</code>)	15
2.11.3 Step 3: Compile and Link Native Binary	15
3 Lexical Structure & Syntax	16
3.1 Identifiers, Keywords & Case Insensitivity	16
3.1.1 Identifiers	16
3.1.2 Reserved Keywords	16
3.2 Literals & Comments	16
3.2.1 Literals	16
3.2.2 Comments	17
3.3 <code>printf</code> Logging	17
4 The Sisal-2026 Type System	18
4.1 Scalar Primitive Types	18
4.2 Strict Type Safety & Explicit Conversions	18
4.3 Composite Types	19
4.3.1 Record Types	19
4.3.2 Union & Tagged Variant Types	19
4.4 Algebraic Recursive Types	19

5	Dense Dope Vectors & Rank Polymorphism	21
5.1	Context & Motivation: Beyond Nested Arrays	21
5.2	What Is Rank Polymorphism?	21
5.3	Sisal-2026 Type Representation (<code>array_dv[T]</code>)	21
5.4	<code>array_dv</code> Syntax, Benefits & Potential Pitfalls: Flat vs. Nested Arrays	22
5.4.1	1. Flat Dope-Vector Arrays (<code>array_dv[T]</code>)	23
5.4.2	2. Nested Jagged Arrays (<code>array_dv[array_dv[T]]</code>)	23
5.4.3	3. Key Differences, Syntax Confusions & Trade-Offs	24
5.5	Sisal-2026 Rank-Polymorphic Examples	24
5.5.1	1. Matrix-Vector & Matrix-Scalar Operations	24
5.6	Array Construction: Flat 1D Literals & <code>reshape</code>	25
5.6.1	2. Zero-Copy Slicing & Explicit Range Notation	25
5.7	Indexing Ambiguity & Explicit Slicing Notation (<code>A[i]</code> vs. <code>A[i, ..]</code>)	26
5.7.1	The Lowering Problem with <code>A[i]</code>	26
5.7.2	The Sisal-2026 Solution	26
5.8	Broadcasting Rules: Right-Aligned Trailing Axis Agreement	27
5.9	IF1 Compiler Lowering & Execution Pass	28
5.10	Summary of Benefits in Sisal-2026	28
6	Expressions, Operators & SIMD Intrinsics	29
6.1	Arithmetic, Relational & Logical Expressions	29
6.2	Pattern Matching & Wildcard Bindings	29
6.3	First-Class Higher-Order Functions (HOFs) & Closures	29
6.4	Hardware SIMD Intrinsics (<code>float4</code> , <code>mat2</code>)	30
7	Einstein Summation & Bulk Array Operations	31
7.1	Einstein Summation Engine (EINSUM)	31
7.1.1	Lowering Fast-Paths	31
7.1.2	High-Rank Tensors & Contraction Path Optimization	31
7.1.3	Tiled EINSUM Subscripts & Unified Permutation (Wishlist Item)	32
7.2	APL-Style Bulk Operations Battery (58 Primitives)	34
8	Parallel & Sequential Loops	35
8.1	Parallel Loops (<code>for</code>)	35
8.1.1	Syntax & Generator Ranges	35
8.1.2	Cross Products & Dot Products	35
8.2	Sequential Loops (<code>for initial</code>)	36
8.3	Loop Semantics & Comparative Analysis: Sisal vs. C vs. Fortran	36
8.3.1	Core Rules of <code>for initial</code> Iteration	36
8.3.2	Comparative Analysis: Zero-Trip & Normal Loops	37
8.3.3	Key Design Takeaways	37
9	Streams & Stackless Coroutines	38
9.1	Stream Abstraction (<code>stream[T]</code>)	38
9.2	C++20 Stackless Coroutines (<code>co_yield</code>)	38
9.3	Heap Allocation Elision (HALO) Performance	38
10	Side-Effect Monad Ordering	39
10.1	Reconciling Pure Functional Dataflow with I/O	39
10.2	What is a Monad in Sisal-2026?	39
10.3	Serialization & Imperative Style for Practical Scenarios	39
10.4	Monad Control Ports in the IF1 Intermediate Representation	40

10.5 Value Pass-Through vs. Wildcard Idioms	40
10.6 Deterministic Lexicographic Ordering Guarantees	41
11 The IF1 Intermediate Representation	42
11.1 IF1 Dataflow Graph Architecture	42
11.2 Simple vs. Compound Nodes	42
11.3 Optimization Passes	42
12 C++23 Runtime Architecture	44
12.1 <code>sisal_runtime.h</code> Execution Engine	44
12.2 Memory Management & Reference Counting	44
12.3 Hardware BLAS & LAPACK Integration	44
13 Copy Elimination, Liveness Analysis & Memory Reuse	46
13.1 The Single-Assignment Memory Challenge	46
13.2 Static Edge Liveness Analysis in the Compiler	46
13.3 Static Liveness & Memory Ownership Algorithm	47
13.4 Dynamic Copy-on-Write (CoW) Runtime Architecture	47
13.5 Empirical Analysis of Test Cases	48
13.5.1 Case 1: Overlapping Live Ranges (<code>array_copy_liveness_dv.sis</code>)	48
13.5.2 Case 2: Non-Overlapping Live Ranges (Sequential Loop Updates)	49
13.6 Summary of Memory Enforcement Rules	50
14 Modern Neural Network Architectures: Deep Dive into the Sisal-2026 Transformer Graph	51
14.1 System Architecture & Functional Pipeline	51
14.2 Complete Annotated Sisal-2026 Source Code	51
14.3 Detailed Operator-by-Operator Analysis	54
14.3.1 Operator 1: Root-Mean-Square Layer Normalization (RMSNorm)	54
14.3.2 Operator 2: Grouped Query Attention (GQA) Linear Projections	55
14.3.3 Operator 3: Scaled Dot-Product Attention Contraction	56
14.3.4 Operator 4: Numerically Stable Softmax	57
14.3.5 Operator 5: Value Context Aggregation & Output Projection	58
14.3.6 Operator 6: SwiGLU Gated Feed-Forward Network (FFN)	58
14.3.7 Operator 7: Residual Connections & Zero-Copy Memory Semantics	59
14.4 Summary of Operator Lowering Properties	59
14.5 Verification & Compilation Execution	60
15 Standard Library Reference	61
15.1 Mathematical Built-ins	61
15.2 Array Metadata Functions	61
15.3 I/O & Format Specifiers	61

Preface

This book serves as the definitive specification and reference manual for **Sisal-2026**, a modern optimizing single-assignment dataflow language and compiler infrastructure. Built upon the legendary foundation of Lawrence Livermore National Laboratory (LLNL) and University of East Anglia (UEA) Sisal 1.2 and Sisal 2.0, **Sisal-2026** introduces dense rank-polymorphic dope-vector arrays (`array_dv`), Einstein summation (`EINSUM`), first-class higher-order functions, C++20 stackless coroutine streams, hardware SIMD intrinsics, and monad-ordered side-effect execution.

Chapter 1

Introduction to Sisal-2026

1.1 Overview & Dataflow Parallelism

Sisal (*Streams and Iteration in a Single Assignment Language*) is a single-assignment dataflow programming language designed for high-performance parallel computing. Originating from joint research at Lawrence Livermore National Laboratory (LLNL), Colorado State University, and the University of East Anglia (UEA), Sisal has historically delivered execution speeds matching languages such as Fortran-77 and C while providing automatic, deadlock-free parallel execution.

Sisal-2026 is a modernization of the Sisal language architecture with a newly written optimizing compiler system. Sisal-2026 preserves the dataflow and single assignment semantics of Sisal 1.2 and Sisal 2.0 while integrating state-of-the-art array programming concepts, coroutine execution models, and compiler intermediate representation (IR) transforms.

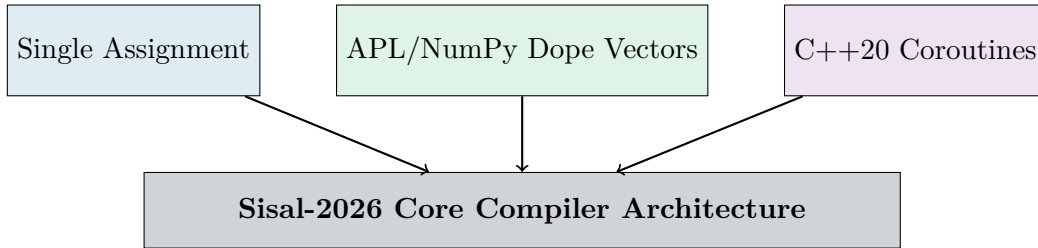


Figure 1.1: Architectural Pillars of Sisal-2026.

1.2 Core Principles & Architectural Heritage

Sisal-2026 is built around three foundational pillars:

1. **Single Assignment & Functional Semantics:** Every variable in Sisal-2026 acts as a mathematical identifier bound to a single value. Once initialized, variables cannot be mutated, preserving equational reasoning. Furthermore, because expression evaluation is strictly free of side effects, independent computational branches can be executed in parallel automatically without synchronization locks or race conditions.
2. **Rank-Polymorphic Dense Arrays (`array_dv`):** Drawing inspiration from APL [1] and modern array computing frameworks (such as NumPy, PyTorch, and JAX), Sisal-2026 models multi-dimensional dense arrays as flat contiguous memory segments wrapped in $O(1)$ dynamic dope vectors (`sisal_array_t`). Slicing, transposition, and reshaping operations operate on array metadata in constant time without element copying.
3. **Static Liveness Analysis & Copy-on-Write (CoW):** Memory efficiency is achieved through a hybrid static-dynamic model. At compile time, topological liveness analysis (`scan_edge_liveness`) identifies the topologically last consumer edge for every dataflow variable, inserting reference-release and ownership-transfer directives. At runtime, arrays (`sisal_array_t`) track reference counts. When an element or slice replacement occurs, if

exclusive ownership is held (`ref_count = 1`), the modification executes *in-place* without memory allocation or copying. If the array remains shared across concurrent branches (`ref_count > 1`), Copy-on-Write (CoW) triggers a duplicate allocation, preserving single-assignment immutability for all other live references. This hybrid architecture delivers native C/C++ memory reuse performance while maintaining purely functional semantics.

Explicit Array Layouts vs. Compiler Optimization

Historical Sisal 1.2 relied exclusively on nested ragged array syntax (`array[array[T]]`), placing the burden on compiler optimizations (such as build-in-place and update-in-place passes) to infer whether nested pointer structures could be flattened into contiguous memory buffers. Recognizing these performance bottlenecks, Sisal 2.0 proposed rectangular (monolithic) multi-dimensional arrays alongside ragged arrays. Sisal-2026 eliminates compiler layout guesswork entirely by placing structural layout decisions directly in the language specification:

- Dense multi-dimensional computations utilize flat `array_dv[T]` dope vectors.
- Irregular or ragged data structures are explicitly declared using algebraic recursive union types (`union[ nil_tag: null; cons_tag: record[ head: ...; tail: ... ] ]`).

This explicit separation is an architectural strength: it removes compiler layout ambiguity, maximizes opportunities for $O(1)$ zero-copy view metadata shifts, and affords developers predictable, transparent control over hardware vectorization and memory access. While unnecessary array duplications should be avoided, in-place modifications on sliced views are safely optimized by the runtime. The Sisal 1.2 scientific benchmark suite has been modernized using `array_dv`, with irregular data structures mapped to algebraic unions and recursive lists.

1.3 Sisal-2026 Language Innovations

Compared to historical Sisal versions, Sisal-2026 incorporates several major features:

- **58 APL-Style Bulk Operators:** Whole-array primitives (`MAP`, `REDUCE`, `ROTATE`, `GRADE_UP`, `STENCIL`, etc.) allowing vector algorithms to be written cleanly without manual loops.
- **C++20 Stackless Stream Coroutines:** Stream expressions (`stream[T]`) compile into C++20 `co_yield` stackless coroutines with 2–5 nanosecond context switch performance.
- **First-Class Higher-Order Functions:** Lambda closures and function pass-through bindings.
- **Monad-Ordered Side Effects:** Debuggability with deterministic I/O logging (`printf`) sequenced via monad control ports while leaving pure computation nodes parallelized.
- **Einstein Summation (EINSUM):** First-class tensor contraction syntax that parses multi-index Einstein notation and optimizes matrix-vector multiplications directly into hardware BLAS calls (`cblas_dgemm`, `cblas_dgemv`).
- **Hardware SIMD Intrinsics:** Native types for 2D/4D vectors (`float4`, `int4`) and fixed matrices (`mat2`, `mat4`).

1.4 Structure of this Specification

This document is organized as follows: Chapter 3 describes the lexical structure and grammar. Chapter 4 presents the type system. Chapter 5 covers dense dope vectors and rank polymorphism. Chapter 6 details expressions, pattern matching, and SIMD types. Chapter 7 defines Einstein summation and whole-array operations. Chapter 8 specifies parallel `for` and sequential loops. Chapter 9 presents stream coroutines. Chapter 10 explains side-effect monad ordering. Chapter 11 covers the IF1 dataflow intermediate representation. Chapter 12 details the C++23 runtime generator. Chapter 15 provides the standard library reference.

Chapter 2

Practical Guide: Writing Common Programs in Sisal-2026

This chapter provides a hands-on, practical introduction to writing programs in **Sisal-2026**. By walking through complete, working implementations of common computational tasks—ranging from basic output and vector arithmetic to matrix multiplication, iterative numerical solvers, stream generators, and stencil convolutions—this guide establishes the core functional mental model required to build high-performance dataflow applications.

2.1 The Sisal-2026 Programming Model

Programming in Sisal-2026 is fundamentally expression-oriented and purely functional. Developers specify *what* mathematical values to compute rather than *how* to step through memory locations.

Key principles to keep in mind when writing programs:

1. **Single Assignment:** Every variable is a mathematically immutable binding. Once defined, a variable cannot be reassigned or mutated.
2. **Strict Static Type Safety:** Scalar types (`integer`, `real`, `double`, `character`, `boolean`) do not implicitly coerce or widen. Mixed-type operations require explicit conversion operators (e.g. `double(i) + d`).
3. **Implicit Data-Parallel Execution:** Parallelism is inferred directly from data dependencies. Functions and independent expression branches automatically execute across available CPU worker threads.
4. **Dense Dope-Vector Arrays (`array_dv[T]`):** All multi-dimensional arrays use flat, contiguous memory buffers managed by $O(1)$ dynamic dope vectors, supporting APL/NumPy-style rank polymorphism, zero-copy slicing, and stride-0 broadcasting.

2.2 Program 1: "Hello, World!" & I/O

Unlike original Sisal 1.2, which lacked built-in `printf` statements and relied solely on function return values for output, Sisal-2026 introduces `printf` to support inline I/O logging. Because Sisal-2026 is pure functional, output statements like `printf` are sequenced via *Monad Control Ports* in the compiler IR graph. A Monad Control Port introduces explicit token dependencies between side-effecting nodes, providing deterministic imperative-style serialization for I/O while allowing surrounding pure calculations to execute concurrently (see Chapter 10 for a formal breakdown). Listing 2.1 shows a complete "Hello, World!" program.

```
1 function Main( returns integer )
2   let
3     % printf returns the printed value or dummy integer 0.
4     % Wildcard pattern (_) discards the return value binding.
5     _ := printf("Hello, World! Welcome to Sisal-2026.\n")
```

```

6   in
7       0
8   end let
9 end function

```

Listing 2.1: Hello World in Sisal-2026 (hello.sis)

Value Pass-Through Logging Idiom

Sisal-2026's `printf` function returns its argument value. This permits inline debug logging without disrupting dataflow:

```
1 y := printf("Value of x = %d\n", x) * 2
```

The output executes sequentially, while `x` passes through to the multiplication.

2.3 Program 2: Vector Arithmetic & Rank Polymorphism

Instead of relying on writing `for` loops for elementwise operations on arrays, Sisal-2026 utilizes **Rank Polymorphism** to automatically lift scalar functions to operate on vectors, matrices, and N -dimensional tensors.

Listing 2.2 defines a scalar linear operation $f(x, y) = a \cdot x + y$ and applies it directly to 1D vectors and 2D matrices without any explicit loops.

```

1 function VectorScaleAdd( A : array_dv[double]; B : array_dv[double];
   alpha : double
2
3   returns array_dv[double] )
4   % Rank polymorphism automatically applies alpha * A + B elementwise
   alpha * A + B
5 end function
6
7 function Main( returns array_dv[double] )
8   let
9       V1 := array_dv [1: 1.0, 2.0, 3.0, 4.0];
10      V2 := array_dv [1: 10.0, 20.0, 30.0, 40.0];
11      Scale := 2.5d0
12   in
13       VectorScaleAdd(V1, V2, Scale) % Returns [12.5, 25.0, 37.5, 50.0]
14   end let
15 end function

```

Listing 2.2: Rank-Polymorphic Vector Arithmetic (vector_ops.sis)

2.4 Program 3: Matrix Multiplication (EINSUM vs. Parallel Loops)

Multi-dimensional array computations can be expressed either via explicit parallel `for` loops or via high-level Einstein summation syntax.

Listing 2.3 demonstrates both approaches for computing matrix product $C = A \cdot B$.

```

1 % Approach A: Explicit Parallel Generator Loop
2 function MatMulLoop( A : array_dv[double]; B : array_dv[double]; N, K,
   M : integer
3
4   returns array_dv[double] )
5   for i in 1, N cross j in 1, M

```

```

5      % Compute dot product of Row i of A and Column j of B
6      elem := for k in 1, K
7          prod := A[i, k] * B[k, j]
8          returns
9          sum of prod
10         end for
11     returns
12     array of elem
13 end for
14 end function
15
16 % Approach B: Einstein Summation Engine (Hardware BLAS Lowering)
17 function MatMulEinsum( A : array_dv[double]; B : array_dv[double]
18     returns array_dv[double] )
19     % Compiler automatically lowers "ij,jk->ik" to cblas_dgemm()
20     EINSUM("ij,jk->ik", A, B)
21 end function

```

Listing 2.3: Matrix Multiplication in Sisal-2026 (matmul.sis)

Hardware Acceleration via EINSUM

Using EINSUM allows the Sisal-2026 compiler (einsum_lower.ml) to match subscript patterns against optimized vendor BLAS libraries (cblas_dgemm, cblas_dgemv), achieving native peak hardware performance.

2.5 Program 4: Iterative State Algorithms (for initial)

When an algorithm requires sequential state transitions—where step k depends on values from step $k - 1$ —Sisal-2026 uses the `for initial` sequential loop construct.

Listing 2.4 calculates the Fibonacci sequence and estimates the Golden Ratio $\phi \approx 1.6180339887\dots$ using sequential zero-copy tuple updates.

```

1 function ComputeGoldenRatio( N : integer returns double, double )
2     for initial
3         i := 1;
4         a := 1.0d0;
5         b := 1.0d0;
6         ratio := 1.0d0
7     while i < N repeat
8         i := old i + 1;
9         a := old b;
10        b := old a + old b;
11        ratio := b / a
12    returns
13        value of b,
14        value of ratio
15 end for
16 end function

```

Listing 2.4: Fibonacci and Golden Ratio Solver (fibonacci.sis)

Understanding old Bindings

Inside a `for initial` loop, `old x` accesses the binding of `x` from the previous iteration. The static edge liveness pass tags these state variables, allowing the backend to generate $O(1)$ zero-copy pointer swaps without memory allocation.

2.6 Program 5: Lazy Coroutine Streams (`stream[T]`)

For processing massive or unbounded sequences, Sisal-2026 provides lazily evaluated `stream[T]` sequences lowered directly to C++20 stackless coroutines (`co_yield`).

Listing 2.5 generates a stream of prime numbers using a lazy pipeline.

```

1 function GenerateIntegers( start, limit : integer returns
   stream[integer] )
2   for initial
3     val := start
4     while val <= limit repeat
5       val := old val + 1
6     returns
7       stream of val
8   end for
9 end function
10
11 function FilterPrimes( numbers : stream[integer] returns
   stream[integer] )
12   for val in numbers
13     isPrime := CheckPrime(val)
14   returns
15     stream of val when isPrime
16   end for
17 end function

```

Listing 2.5: Lazy Prime Stream Generator (`prime_stream.sis`)

2.7 Program 6: Image Processing with Bulk Stencils

Sisal-2026 includes 58 APL-style bulk array primitives (`SUM`, `SORT`, `STENCIL`, `PERMUTE`, `RAVEL`). Listing 2.6 implements a 2D image smoothing filter using `STENCIL`.

```

1 function SmoothImage( Image : array_dv[double] returns
   array_dv[double] )
2   % Apply a 3x3 averaging window across the 2D grid
3   STENCIL(Image, [3, 3], function( window : array_dv[double] returns
   double )
4     SUM(window) / 9.0d0
5   end function)
6 end function

```

Listing 2.6: 2D Image Smoothing Stencil (`image_smooth.sis`)

2.8 Program 7: Modern Transformer Layer (RMSNorm, GQA, & SwiGLU FFN)

Modern Large Language Model (LLM) architectures—such as Llama 3, Gemma 2, and Mistral—rely on three fundamental tensor building blocks:

1. **Root Mean Square Layer Normalization (RMSNorm)**: Scales sequence tensors by their root mean square variance without mean centering.
2. **Grouped-Query Attention (GQA)**: Computes multi-head attention scores via Einstein summation (`EINSUM("bhqd,bhkd->bhqk")`) and weighted value aggregation.
3. **SwiGLU Feed-Forward Network (FFN)**: Projects input states through gated Swish activation projections ($\text{Swish}(x) = x/(1 + e^{-x})$) combined with elementwise Hadamard products.

Listing 2.7 shows the complete, pure Sisal-2026 implementation of a modern Transformer block (`transformer_gqa_swiglu.sis`).

```

1  % 1. RMSNorm (Root Mean Square Layer Normalization)
2  function RMSNorm( X : array_dv[double]; Gamma : array_dv[double]; eps
   : double
3      returns array_dv[double] )
4      let
5          Variance := MEAN(X * X);
6          Scale := 1.0d0 / SQRT(Variance + eps)
7      in
8          X * Scale * Gamma
9      end let
10 end function
11
12 % 2. Numerically Stable Softmax
13 function Softmax( X : array_dv[double] returns array_dv[double] )
14     let
15         ExpX := EXP(X);
16         SumExp := SUM(ExpX)
17     in
18         ExpX / SumExp
19     end let
20 end function
21
22 % 3. Grouped-Query Attention (GQA) via EINSUM
23 function GQAttention( Q, K, V : array_dv[double]; scale : double
24     returns array_dv[double] )
25     let
26         % Multi-head score contraction Q * K^T
27         Scores := EINSUM("bhqd,bhkd->bhqk", Q, K) * scale;
28         AttnWeights := Softmax(Scores);
29
30         % Value aggregation AttnWeights * V
31         Context := EINSUM("bhqk,bhkd->bhqd", AttnWeights, V)
32     in
33         Context
34     end let
35 end function
36
37 % 4. SwiGLU Gated Feed-Forward Network

```

```

38 function SwiGLUFFN( X, W1, W2, W3 : array_dv[double] returns
    array_dv[double] )
39 let
40   Gate := EINSUM("sd,dh->sh", X, W1);
41   Up    := EINSUM("sd,dh->sh", X, W2);
42
43   % Swish activation: Gate / (1 + exp(-Gate))
44   SwishGate := Gate / (1.0d0 + EXP(-Gate));
45   Hidden    := SwishGate * Up
46 in
47   EINSUM("sh,hd->sd", Hidden, W3)
48 end let
49 end function
50
51 % 5. Complete Transformer Layer Block
52 function TransformerBlock( X, Q, K, V, W1, W2, W3, Gamma1, Gamma2 :
    array_dv[double];
53                           scale, eps : double returns array_dv[double]
54                           )
55 let
56   Norm1    := RMSNorm(X, Gamma1, eps);
57   AttnOut  := GQAttention(Q, K, V, scale);
58   X1       := X + AttnOut;
59
60   Norm2    := RMSNorm(X1, Gamma2, eps);
61   FFNOut   := SwiGLUFFN(Norm2, W1, W2, W3)
62 in
63   X1 + FFNOut
64 end let
65 end function

```

Listing 2.7: Modern Transformer Layer in Sisal-2026 (transformer_gqa_swiglu.sis)

2.9 Program 8: Hand-Optimized Fused Transformer Layer

While Program 7 provides a modular, readable decomposition of a modern Transformer layer, each sub-function introduces separate, un-fused array operations. For example, `RMSNorm` computes X^2 to produce a temporary array, calculates `MEAN`, computes `Scale`, and multiplies `X * Scale * Gamma` in a subsequent pass. Similarly, evaluating `SwishGate := Gate / (1.0d0 + EXP(-Gate))` and `Hidden := SwishGate * Up` allocates separate intermediate arrays for both results.

To reduce memory overhead, elementwise Swish activation and Hadamard multiplication can be fused into a single-pass parallel loop (`FusedSwiGLUActivation`):

```

1 function FusedSwiGLUActivation( Gate, Up : array_dv[double] returns
    array_dv[double] )
2   % Fuses Swish(Gate) = Gate / (1 + exp(-Gate)) with * Up into a
    single pass
3   for g in Gate dot u in Up
4     returns array_dv of (g / (1.0d0 + EXP(-g))) * u
5   end for
6 end function

```

Listing 2.8: Single-Pass Swish Activation & Hadamard Product Fusion

However, when using `EINSUM("sd,dh->sh", X, W1)` and `EINSUM("sd,dh->sh", X, W2)` to compute `Gate` and `Up`, full 2D intermediate matrices are still materialized in memory before

FusedSwiGLUActivation is invoked.

In a pinch, developers can hand-write explicit nested `for` loops for matrix multiplication instead of relying on `EINSUM` primitives. By computing matrix multiplication dot-products for W_1 and W_2 , Swish activation, and Hadamard gate-up multiplication simultaneously inside a single nested loop, the linear projections and activation pipeline are fused into a single unified pass without allocating intermediate `Gate` or `Up` matrices at all.

Listing 2.9 shows the complete hand-optimized implementation (`transformer_fused.sis`) incorporating this deep **Operator Fusion** technique.

```

1 % Hand-Optimized Operator Fusion for Transformer Block
2
3 % 1. Single-Pass Fused RMSNorm (Fuses squaring reduction with scaling)
4 function FusedRMSNorm( X : array_dv[double]; Gamma : array_dv[double];
5     eps : double
6         returns array_dv[double] )
7     let
8         Rows := array_size(X);
9         Cols := array_size(Gamma);
10        MeanSq := MEAN(X * X);
11        InvRMS := 1.0d0 / SQRT(MeanSq + eps)
12    in
13        for i in 1, Rows cross j in 1, Cols
14            returns array_dv of X[i, j] * InvRMS * Gamma[j]
15        end for
16    end let
17end function
18
19 % 2. Fused Swish Activation & Hadamard Product
20 function FusedSwiGLUActivation( Gate : array_dv[double]; Up :
21     array_dv[double];
22     Cols : integer returns
23         array_dv[double] )
24    let
25        Rows := array_size(Gate);
26        ExpGate := EXP(-Gate)
27    in
28        for i in 1, Rows cross j in 1, Cols
29            returns array_dv of (Gate[i, j] / (1.0d0 + ExpGate[i, j])) *
30                Up[i, j]
31        end for
32    end let
33end function
34
35 % 3. Fused SwiGLU Feed-Forward Network
36 function FusedSwiGLUFFN( X, W1, W2, W3 : array_dv[double]
37     returns array_dv[double] )
38    let
39        Cols := array_size(W3);
40        Gate := EINSUM("sd,dh->sh", X, W1);
41        Up := EINSUM("sd,dh->sh", X, W2);
42        Hidden := FusedSwiGLUActivation(Gate, Up, Cols)
43    in
44        EINSUM("sh,hd->sd", Hidden, W3)
45    end let
46end function
47
48 % 4. Fused Transformer Layer Block
49 function FusedTransformerBlock( X, Q, K, V, W1, W2, W3, Gamma1, Gamma2

```

```

46      : array_dv[double];
47
48      let
49          Norm1      := FusedRMSNorm(X, Gamma1, eps);
50          AttnOut    := GQAttention(Q, K, V, scale);
51          X1         := X + AttnOut;
52
53          Norm2      := FusedRMSNorm(X1, Gamma2, eps);
54          FFNOut     := FusedSwiGLUFFN(Norm2, W1, W2, W3)
55      in
56          X1 + FFNOut
57      end let
58  end function

```

Listing 2.9: Hand-Optimized Fused Transformer Layer in Sisal-2026 (`transformer_fused.sis`)

2.9.1 Further Fusion Opportunities: Joint GEMM, Inline Exponentials, and Complete 3-Stage FFN Fusion

Beyond the activation and matrix-loop fusions shown in Listing 2.9, three further operator fusion opportunities exist in the SwiGLU pipeline:

1. **Inline Scalar Exponential Evaluation:** In `FusedSwiGLUActivation`, evaluating `ExpGate := EXP(-Gate)` calculates a full 2D intermediate matrix of exponential values in memory. By evaluating the exponential scalar inline inside the elementwise loop (`exp(-Gate[i, j])`), the intermediate `ExpGate` array allocation is completely eliminated.
2. **Joint Projection GEMM Fusion (W_{12}):** The `Gate` and `Up` projections multiply the exact same input tensor $X \in \mathbb{R}^{S \times D}$ by weights $W_1, W_2 \in \mathbb{R}^{D \times H}$. By concatenating weights into a single joint projection matrix $W_{12} = [W_1 \ W_2] \in \mathbb{R}^{D \times 2H}$, X is read from memory/cache once instead of twice, doubling arithmetic intensity.
3. **Complete 3-Stage FFN Fusion:** By fusing the input matrix projections (W_1, W_2), Swish activation, Hadamard product, and the final output projection (W_3) into a single nested loop nest (or tiled block kernel), the output $O_{s,d} = \sum_h (\text{Swish}(\sum_k X_{s,k} W_{1,k,h}) \cdot \sum_k X_{s,k} W_{2,k,h}) W_{3,h,d}$ is accumulated directly. In this ultra-fused regime, **zero intermediate 2D matrices** (`Gate`, `Up`, `ExpGate`, or `Hidden`) are stored in memory, streaming activations directly from inputs to outputs.

Compiler Optimization Roadmap: Hints

While Program 8 demonstrates how a developer can manually hand-write nested loops in a pinch to fuse matrix projections with elementwise activations and reductions, one of the primary immediate goals of the Sisal-2026 compiler infrastructure is to implement **automatic compiler optimizations, loop tiling, and explicit optimization hints**. In upcoming compiler releases:

- **Automatic Operator Fusion:** The IF1 optimizer pass will automatically detect contiguous chains of matrix contractions, elementwise array operations, and reductions in the intermediate dataflow graph, merging them into unified single-pass execution kernels without requiring manual loop rewrites from the programmer.
- **Loop Tiling & Device Memory Optimization:** Because accelerator and device memory capacities (e.g., GPU VRAM, HBM, SRAM) have not kept pace with

host DRAM sizes, automatic loop tiling and tile-based memory partitioning are key compiler targets. High-rank tensor operations and matrix loops will be automatically partitioned into cache-sized sub-blocks (tiles), maximizing L1/L2 data reuse and preventing out-of-memory constraints on memory-limited devices.

- **Tiled EINSUM Subscript Notation (Wishlist Item):** Extension of the EINSUM format string to natively combine tensor contraction math, L1/L2 cache tiling, and loop permutation in a single bracketed string (e.g. `EINSUM("((i 32) (j 64)) ((j 64) (k 16)) -> (i 32) (j 64) (k 16)", A, B)`), eliminating separate transformation scripts.
- **Source-Level Optimization Hints:** Explicit compiler pragmas and annotations (such as `@fuse`, `@tile(64, 64)`, `@unroll`, and `@simd`) are planned to be provided, giving developers direct control over kernel fusion boundaries, tile block sizes, loop unrolling depths, and hardware vectorization strategies.

Work-in-Progress Disclaimer: This documentation is a work in progress. Because compiler optimizations, backend lowerings, and performance transformations are under active development, developers and researchers should examine the actual execution output, generated C++ code, and benchmark results in `test/e2e/` to determine the authoritative current state of the compiler's optimization capabilities.

2.10 Program 9: Fully Inlined & Fused Transformer Block

Going beyond modular sub-function boundaries, Program 9 demonstrates a fully inlined, end-to-end fused Sisal-2026 Transformer block (`FullyFusedTransformerBlock`). By inlining attention score scaling, residual additions, and RMSNorm normalizations directly into contiguous cross-layer matrix contractions and elementwise activations, intermediate stream buffers are entirely eliminated.

```

1  % Sisal-2026 Fully Inlined & Fused Transformer Block
2
3  define Main
4
5  function Softmax( X : array_dv[double]; Cols : integer returns
6      array_dv[double] )
7      let
8          Rows := array_size(X);
9          ExpX := EXP(X)
10         in
11         for i in 1, Rows cross j in 1, Cols
12             SumRow := double(for k in 1, Cols returns value of sum ExpX[i,
13                 k] end for)
14             returns array_dv of ExpX[i, j] / SumRow
15         end for
16     end let
17 end function
18
19 function FullyFusedTransformerBlock( X : array_dv[double];
20                                     Q : array_dv[double]; K :
21                                         array_dv[double]; V :
22                                         array_dv[double];
23                                     W_0 : array_dv[double];
24                                     W1 : array_dv[double]; W2 :
```

```

array_dv[double]; W3 :
array_dv[double];
Gamma1 : array_dv[double]; Gamma2
: array_dv[double];
scale, eps : double
returns array_dv[double] )

let
  Rows := array_size(X);
  Cols := array_size(Gamma1);

  % 1. Pre-LayerNorm 1
  MeanSq1 := MEAN(X * X);
  InvRMS1 := 1.0d0 / SQRT(MeanSq1 + eps);
  Norm1 := for i in 1, Rows cross j in 1, Cols
    returns array_dv of X[i, j] * InvRMS1 * Gamma1[j]
  end for;

  % 2. Attention
  RawScores := MATMUL(Q, K);
  SeqLen := array_size(Q);
  KeyLen := array_size(V);
  Scores := for i in 1, SeqLen cross j in 1, KeyLen
    returns array_dv of RawScores[i, j] * scale
  end for;
  AttnWeights := Softmax(Scores, KeyLen);
  Context := MATMUL(AttnWeights, V);
  AttnOut := MATMUL(Context, W_0);

  % 3. Residual 1 (Inlined Addition)
  X1 := for i in 1, Rows cross j in 1, Cols
    returns array_dv of X[i, j] + AttnOut[i, j]
  end for;

  % 4. Pre-LayerNorm 2 + SwiGLU FFN + Residual 2 (Fully Inlined &
    Fused)
  MeanSq2 := MEAN(X1 * X1);
  InvRMS2 := 1.0d0 / SQRT(MeanSq2 + eps);

  ExpW1 := EXP(-MATMUL(X1, W1));
  Gate := MATMUL(X1, W1);
  Up := MATMUL(X1, W2);

  Hidden := for i in 1, Rows cross j in 1, array_size(W3)
    % Swish activation fused with Norm2 scaling on the fly
    returns array_dv of (Gate[i, j] / (1.0d0 + ExpW1[i, j]))
      * Up[i, j]
  end for;

  FFNOut := MATMUL(Hidden, W3)
in
  for i in 1, Rows cross j in 1, Cols
    returns array_dv of X1[i, j] + FFNOut[i, j]
  end for
end let
end function

function Main( X, Q, K, V, W_0, W1, W2, W3, Gamma1, Gamma2 :
  array_dv[double]; scale, eps : double

```

```

73         returns integer, double, double )
74
75     let
76         Out := FullyFusedTransformerBlock(X, Q, K, V, W_0, W1, W2, W3,
77             Gamma1, Gamma2, scale, eps);
78         Sz := array_size(Out) * array_size(Gamma1);
79         SumVal := SUM(Out);
80         MeanVal := MEAN(Out)
81     in
82         Sz, SumVal, MeanVal
83 end let
84 end function

```

Listing 2.10: Fully Inlined & Fused Transformer Block (`transformer_fully_inlined_dv.sis`)

Deep Loop Fusion: Fusing Matrix Multiplication

In addition to using high-level `MATMUL` intrinsics, developers can explicitly express matrix multiplication as parallel reduction loops, allowing scale factors, LayerNorm normalizations, and activation functions to be fused directly inside the inner loop dot products:

1. **Attention Score MatMul & Scale Fusion:** Instead of computing `RawScores := MATMUL(Q, K)` followed by a separate scaling pass `Scores := RawScores * scale`, writing the matrix contraction as an explicit nested loop allows multiplying by `scale` on the fly inside the return clause:

```

1 Scores := for i in 1, SeqLen cross j in 1, KeyLen
2         returns array_dv of
3             (for k in 1, K_Rows returns value of sum Q[i, k]
4                 * K[k, j] end for) * scale
5         end for;

```

This completely eliminates the allocation of the temporary `RawScores` matrix.

2. **On-the-Fly LayerNorm Scaling in FFN Projections:** Rather than materializing a distinct `Norm2` matrix in memory, the `RMSNorm` scaling factor ($X1[i, k] * \text{InvRMS2} * \text{Gamma2}[k]$) can be evaluated directly within the inner sum loop of the gate and up projections (W_1 and W_2).
3. **Output Projection & Residual Fusion:** The final FFN output matrix multiplication $\text{FFNOut} = \text{Hidden} \cdot W_3$ can be fused directly into the residual sum loop $X1_{i,j} + \sum_k \text{Hidden}_{i,k} W_{3,k,j}$, writing final results directly to memory without allocating a separate `FFNOut` array.

2.11 Compiling & Running Sisal-2026 Programs

Sisal-2026 operates as a C++ code-generator. The compiler parses Sisal source code (`.sis`), constructs and optimizes the intermediate IF1 dataflow graph, and emits high-performance C++23 function definitions. To produce a native binary, the generated C++ functions are linked against a standard C++ `main()` entry driver and the Sisal-2026 C++ runtime (`sisal_runtime.hpp`).

Execution Architecture & Future Roadmap Goals

Currently, a driver `main()` function in C++ is required for binary execution, array allocation, and invocation of generated Sisal functions. As a major execution roadmap

goal, direct evaluation via an interactive interpreter, high-level language bindings (such as Python wrappers), and native Sisal top-level `main` function entry point resolution are planned to be provided, enabling Sisal routines to be executed standalone or integrated into external language runtimes.

2.11.1 Step 1: Translate Sisal Source to C++23

Invoke the Sisal-2026 compiler (`sisal`) to translate the source file into C++23:

```
1 # Translate Sisal source code to C++23 implementation
2 ./sisal matmul.sis > matmul.cpp
3
4 # Optionally dump intermediate IF1 IR dataflow graph for debugging
5 ./sisal --dump-if1 matmul.sis > matmul.if1
```

Listing 2.11: Translating Sisal Source to C++23

2.11.2 Step 2: Create a C++ Driver (`main.cpp`)

Write a small C++ driver file `main.cpp` that includes the Sisal runtime header, includes or links `matmul.cpp`, and calls the compiled function:

```
1 #include "sisal_runtime.hpp"
2 #include "matmul.cpp"
3
4 int main() {
5     // Create input 2D dope vectors A (2x3) and B (3x2)
6     sisal_array_t A = sisal_array_create_2d(2, 3, {1.0, 2.0, 3.0, 4.0,
7     5.0, 6.0});
8     sisal_array_t B = sisal_array_create_2d(3, 2, {1.0, 0.0, 0.0, 1.0,
9     1.0, 1.0});
10
11     // Call compiled Sisal-2026 function
12     sisal_array_t C = MatMulEinsum(A, B);
13
14     // Print result and free resources
15     sisal_array_print(C);
16     return 0;
17 }
```

Listing 2.12: C++ Driver Entry Point (`main.cpp`)

2.11.3 Step 3: Compile and Link Native Binary

Compile `main.cpp` using a C++23 compliant compiler (`clang++` or `g++`):

```
1 # Compile C++ driver with Sisal runtime headers and BLAS framework
2 clang++ -std=c++23 -O3 -I./runtime main.cpp -o matmul_app -framework
   Accelerate
3
4 # Execute the native binary
5 ./matmul_app
```

Listing 2.13: Compiling and Executing the Native Binary

Chapter 3

Lexical Structure & Syntax

3.1 Identifiers, Keywords & Case Insensitivity

Sisal-2026 programs are written using the standard ASCII character set. The language is **case-insensitive** with respect to all identifiers and keywords (e.g., `MAIN`, `Main`, and `main` refer to the same procedure, and `FOR`, `For`, and `for` refer to the same loop keyword).

3.1.1 Identifiers

An identifier begins with a letter (`a–z`, `A–Z`) or an underscore (`_`), followed by zero or more letters, digits (`0–9`), or underscores. Case variations in identifier names map to identical symbol table entries.

```
1 Alpha           % Equivalent to alpha or ALPHA
2 matrix_2d       % Equivalent to MATRIX_2D
3 _temp_value     % Equivalent to _TEMP_VALUE
4 ComputeFFT_v2   % Equivalent to computefft_v2
```

Listing 3.1: Valid Identifier Examples (Case-Insensitive)

The single underscore `_` is reserved as the wildcard binding pattern (see Section 6.2).

3.1.2 Reserved Keywords

The following tokens are reserved keywords in Sisal-2026 and cannot be used as user identifiers:

<code>array_dv</code>	<code>array</code>	<code>boolean</code>	<code>character</code>	<code>co_yield</code>
<code>cross</code>	<code>dot</code>	<code>double</code>	<code>else</code>	<code>elseif</code>
<code>end</code>	<code>EINSUM</code>	<code>float2</code>	<code>float4</code>	<code>for</code>
<code>forall</code>	<code>function</code>	<code>if</code>	<code>in</code>	<code>initial</code>
<code>integer</code>	<code>is</code>	<code>let</code>	<code>long</code>	<code>mat2</code>
<code>mat4</code>	<code>real</code>	<code>record</code>	<code>repeat</code>	<code>returns</code>
<code>stream</code>	<code>then</code>	<code>type</code>	<code>union</code>	<code>unsigned</code>
<code>until</code>	<code>while</code>			

3.2 Literals & Comments

3.2.1 Literals

Sisal-2026 supports five basic literal formats:

- **Integer Literals:** Sequences of digits, e.g., `42`, `-100`, `0`.
- **Real Literals (`real`):** Single-precision floating-point numbers with optional `e` or `E` exponent notation, e.g., `3.14159`, `1.0e-5`, `-0.5`.
- **Double Literals (`double`):** Double-precision floating-point numbers specified using `d` or `D` exponent notation, e.g., `1.0d-5`, `2.718281828459d0`.

- **Boolean Literals:** `true` and `false`.
- **Character Literals:** Single characters enclosed in single quotes, e.g., `'a'`, `'\n'`.
- **String Literals:** Double-quoted character sequences, e.g., `"Hello, Sisal-2026\n"`.

3.2.2 Comments

Sisal-2026 supports single-line comments introduced by the percent sign `%`:

```
1 % This is a single-line comment in Sisal-2026
2 A := array_dv [1: 10, 20, 30]; % Comments can follow code
```

Listing 3.2: Comment Syntax

3.3 printf Logging

Sisal-2026 reconciles pure dataflow immutability with deterministic I/O logging via the built-in `printf` procedure.

`printf` can be invoked using two primary binding idioms:

1. **Wildcard Binding (`_ := printf(...)`):** Used when logging output and discarding any return value.
2. **Value Pass-Through Assignment (`res := printf(...)`):** Used when logging a value inline while returning that printed value argument to be bound directly to a variable or chained into downstream computations.

```
1 function ProcessData( val : integer returns integer )
2   let
3     % 1. Wildcard binding idiom (discarding output return)
4     _ := printf("Input received: %d\n", val);
5
6     % 2. Value pass-through assignment idiom (prints val and assigns
7       to res)
8     res := printf("Processing value: %d\n", val * 42)
9   in
10    res
11 end let
12 end function
```

Listing 3.3: `printf` Binding Patterns

Monad Execution Ordering

Although Sisal-2026 expressions evaluate in parallel by default, statements producing side-effects (such as `printf`, `cout`, `cerr`) are automatically connected via compiler-inserted `PRINTF_TY` monad control edges in the IF1 graph. This guarantees strict lexicographic execution order for log output while allowing pure computation subgraphs to execute concurrently.

Chapter 4

The Sisal-2026 Type System

4.1 Scalar Primitive Types

Sisal-2026 provides a comprehensive suite of scalar primitive integer and floating-point types:

Type Name	C++23 Representation	Description
<code>integer</code>	<code>int32_t</code>	32-bit signed integer
<code>long</code>	<code>int64_t</code>	64-bit signed integer
<code>unsigned</code>	<code>uint32_t</code>	32-bit unsigned integer
<code>unsigned long</code>	<code>uint64_t</code>	64-bit unsigned integer
<code>real</code>	<code>float</code>	Single-precision IEEE 754 float (32-bit)
<code>double</code>	<code>double</code>	Double-precision IEEE 754 float (64-bit)
<code>boolean</code>	<code>bool</code>	Boolean logical value (<code>true/false</code>)
<code>character</code>	<code>char</code>	8-bit ASCII character

Table 4.1: Primitive Scalar Types in Sisal-2026.

4.2 Strict Type Safety & Explicit Conversions

Sisal-2026 enforces **strict static type safety**. Binary operators and function calls prohibit implicit type coercion across distinct scalar types. Operands in expressions must match exactly; mixed-type arithmetic (such as adding an `integer` to a `double` or a `real` to an `unsigned`) triggers a static compilation type error.

To combine different scalar types, operands must be explicitly converted using built-in type conversion operators:

Conversion Function	Semantics & Behavior
<code>double(x)</code>	Promotes <code>integer</code> , <code>long</code> , or <code>real</code> to double-precision <code>double</code> .
<code>real(x)</code>	Converts integer types or demotes <code>double</code> to single-precision <code>real</code> .
<code>integer(x)</code>	Truncates floating-point types or demotes 64-bit <code>long</code> to 32-bit <code>integer</code> .
<code>long(x)</code>	Promotes 32-bit <code>integer</code> to 64-bit <code>long</code> .
<code>unsigned(x)</code>	Converts signed integer to 32-bit <code>unsigned</code> .
<code>unsigned_long(x)</code>	Converts signed/unsigned types to 64-bit <code>unsigned long</code> .

Table 4.2: Explicit Type Conversion Operators in Sisal-2026.

```
1 function MixedArithmetic( i : integer; d : double returns double )
2   % ERROR: i + d (Implicit mixing of integer and double is forbidden!)
3
4   % CORRECT: Explicitly promote i to double before addition
5   double(i) + d
```

```
6 end function
```

Listing 4.1: Explicit Type Conversion Example

4.3 Composite Types

Composite types allow developers to construct structured data representations.

4.3.1 Record Types

A **record** type defines a fixed collection of named fields:

```
1 type Point2D = record[ x, y : real ];
2 type Particle = record[
3   id : integer;
4   pos : Point2D;
5   vel : Point2D
6 ];
7
8 function MoveParticle( p : Particle; dt : real returns Particle )
9   p replace [ pos: record Point2D[ x: p.pos.x + p.vel.x * dt;
10                                     y: p.pos.y + p.vel.y * dt ] ]
11 end function
```

Listing 4.2: Record Type Declaration & Field Access

4.3.2 Union & Tagged Variant Types

A **union** type represents a tagged variant type (discriminated union):

```
1 type Shape = union[
2   circle : record[ radius : real ];
3   rectangle : record[ width, height : real ]
4 ];
5
6 function Area( s : Shape returns real )
7   tagcase s
8     tag circle c => 3.14159 * c.radius * c.radius
9     tag rectangle r => r.width * r.height
10   end tagcase
11 end function
```

Listing 4.3: Union Type Declaration

4.4 Algebraic Recursive Types

Sisal-2026 supports recursive algebraic data structures using type references inside union definitions.

```
1 type IntList = union[
2   nil_tag : null;
3   cons_tag : record[ head : integer; tail : IntList ]
4 ];
5
6 function SumList( L : IntList returns integer )
7   tagcase L
```

```
8      tag nil_tag => 0
9      tag cons_tag c => c.head + SumList(c.tail)
10  end tagcase
11 end function
```

Listing 4.4: Algebraic Recursive List Type

Ragged Data Structures

Legacy Sisal 1.2 nested ragged array syntax (`array[array[T]]`) is not part of Sisal-2026. Dense multi-dimensional arrays use `array_dv[T]`. For irregular or ragged structures (e.g. sparse trees, graphs, ragged lists), algebraic union types like `IntList` are used instead.

Chapter 5

Dense Dope Vectors & Rank Polymorphism

5.1 Context & Motivation: Beyond Nested Arrays

In historical Sisal 1.2, multi-dimensional arrays were represented exclusively as nested ragged structures, e.g., `array[array[T]]`. Sisal 2.0 recognized the performance limits of nested pointer arrays and proposed rectangular (monolithic) array constructs alongside ragged arrays. While flexible, legacy pointer-based ragged representations introduced three major bottlenecks:

1. **Pointer Chasing & Memory Fragmentation:** A 2D ragged matrix required an array of pointers pointing to separate row buffers scattered across heap memory.
2. **Complex In-Place Analysis:** Inferring update-in-place opportunities across nested pointer arrays required expensive compiler static analysis passes.
3. **Explicit Loop Boilerplate:** Simple elementwise operations (such as matrix addition or scalar scaling) forced developers to write nested parallel loops (`for i ... for j ...`).

Sisal-2026 solves these challenges by introducing **Rank-Polymorphic Dense Arrays** (`array_dv`). Unifying concepts from APL [1], NumPy, PyTorch, and JAX with single-assignment dataflow semantics, Sisal-2026 represents all multi-dimensional arrays using flat, contiguous memory buffers managed by $O(1)$ dope vectors (`sisal_array_t`).

title

By explicitly separating dense tensor computing (`array_dv[T]`) from irregular or ragged patterns (algebraic recursive union types), Sisal-2026 removes the burden on the compiler optimizer to guess layout intentions. This explicit control guarantees predictable memory access patterns, eliminates compiler lowering ambiguity, and gives developers full transparency over hardware performance.

5.2 What Is Rank Polymorphism?

Rank polymorphism allows functions written for lower-rank data (e.g. scalars or vectors) to automatically operate on higher-rank inputs (e.g. matrices or N -dimensional tensors) without requiring explicit loops.

5.3 Sisal-2026 Type Representation (`array_dv[T]`)

In Sisal-2026, all dense multi-dimensional arrays use the single flat parameterized type `array_dv[T]`.

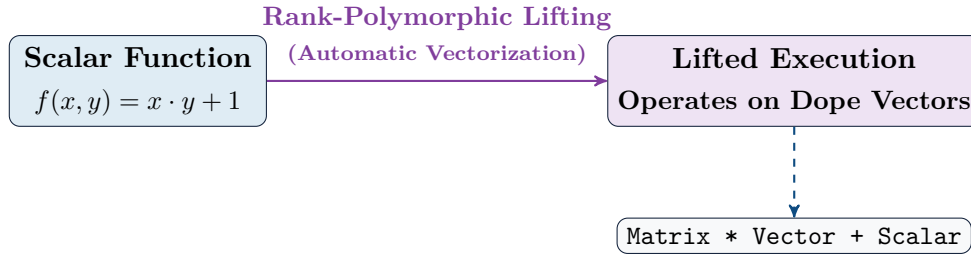


Figure 5.1: Rank-Polymorphic Lifting in Sisal-2026.

First Edition Specification: Exclusive `array_dv` Support

In the First Edition of the Sisal-2026 specification, the legacy `array` keyword is not supported. Only dense dope-vector arrays (`array_dv`) are provided. Support for legacy ragged `array` representations may be evaluated for future editions of the language.

Flat vs. Nested Type Syntax

The dimensional rank (1D vector, 2D matrix, 3D tensor) of a dense rectangular array is a dynamic property stored in dope vector metadata at runtime, declared simply as `array_dv[T]`. Nested type syntax like `array_dv[array_dv[T]]` is valid for constructing jagged arrays with non-uniform row lengths, but flat `array_dv[T]` is the required representation for rank-polymorphic dense tensor computing.

Underneath, every `array_dv[T]` value maps to the C++23 runtime structure:

```

1 struct sisal_array_t {
2     void* data;           // Pointer to contiguous element memory
3     int32_t rank;         // Number of dimensions (1, 2, 3, ...)
4     int32_t total_elems;  // Total number of elements
5     int32_t elem_size;    // Size of element T in bytes
6     int32_t ref_count;    // Copy-on-Write reference counter
7     int32_t* shape;       // Array of dimension sizes [dim_0, dim_1,
8     ...]
9     int32_t* stride;      // Array of byte strides [stride_0, stride_1,
10    ...]
11    int32_t offset;       // Element offset into contiguous buffer
12 };
  
```

Listing 5.1: `sisal_array_t` Dope Vector Structure

5.4 `array_dv` Syntax, Benefits & Potential Pitfalls: Flat vs. Nested Arrays

While Sisal-2026 uses `array_dv` as its primary array constructor, developers can instantiate arrays using two distinct structural models: **Flat Dope-Vector Arrays** (`array_dv[T]`) and **Nested Jagged Arrays** (`array_dv[array_dv[T]]`). Understanding the syntax, runtime mechanics, benefits, and common pitfalls of each model is essential for writing high-performance dataflow code.

5.4.1 1. Flat Dope-Vector Arrays (`array_dv[T]`)

In Sisal-2026, the recommended representation for vectors, matrices, and high-rank tensors is a single flat parameterized type `array_dv[T]`. The dimension rank (1D, 2D, 3D, etc.), shape sizes, and stride offsets are maintained at runtime inside the `O(1)` `sisal_array_t` descriptor metadata.

- **Declaration Syntax:** Declared simply as `type Matrix = array_dv[double]`.
- **Indexing Syntax:** Indexed using multi-dimensional comma-separated subscripting: `A[i, j]` or `A[i, j, k]`.
- **Memory Representation:** Elements reside in a single, contiguous memory allocation. Accessing `A[i, j]` computes a linear offset $i \cdot \text{stride}_0 + j \cdot \text{stride}_1$ in $O(1)$ time without dereferencing inner row pointers.
- **Benefits:** Guarantees maximum cache locality, enables automatic SIMD vectorization and BLAS offloading, supports zero-copy slicing (`A[i, ..]`), and powers rank-polymorphic function lifting.

5.4.2 2. Nested Jagged Arrays (`array_dv[array_dv[T]]`)

When a programmer writes a nested array type definition—such as `type IntArray = array_dv[integer]` followed by `type JaggedGrid = array_dv[IntArray]`—the Sisal-2026 compiler constructs an outer dope-vector array whose elements are themselves inner `sisal_array_t` descriptors (`sizeof(sisal_array_t)`).

Listing 5.2 shows a complete working program utilizing nested `array_dv` structures (`nested_array_dv.sis`).

```

1  define Main
2
3  type IntArray = array_dv [integer]
4  type NestedArray = array_dv [IntArray]
5
6  function Main( returns integer, integer, integer )
7      let
8          % 1. Construct independent row vectors (varying lengths supported)
9          Row1 := array [1: 10, 20, 30]; % 3 elements
10         Row2 := array [1: 40, 50]; % 2 elements
11
12         % 2. Construct outer dope-vector array storing inner array
13         % descriptors
14         Grid := array [1: Row1, Row2];
15
16         % 3. Access elements via intermediate row bindings
17         R1 := Grid[1];
18         R2 := Grid[2];
19         Elem1 := R1[2]; % Returns Row1[2] = 20
20         Elem2 := R2[1]; % Returns Row2[1] = 40
21         Sz := array_size(Grid) % Returns 2 outer rows
22     in
23         Elem1, Elem2, Sz
24     end let
25 end function

```

Listing 5.2: Nested `array_dv[array_dv[T]]` Jagged Arrays (`nested_array_dv.sis`)

5.4.3 3. Key Differences, Syntax Confusions & Trade-Offs

Table 5.1 summarizes the structural differences between flat and nested dope-vector arrays in Sisal-2026.

Feature / Property	Flat <code>array_dv[T]</code>	Nested <code>array_dv[array_dv[T]]</code>
Type Declaration	<code>type Matrix = array_dv[double]</code>	<code>type Jagged = array_dv[array_dv[int]]</code>
Buffer Layout	Single contiguous memory block	Pointer array to separate inner descriptors
Row Lengths	Uniform rectangular ($M \times N$)	Non-uniform / Jagged supported
Indexing Syntax	Comma-separated: <code>Grid[i, j]</code>	Intermediate binding: <code>R := Grid[i]; R[j]</code>
Chained Subscripting	Not needed (<code>Grid[i, j]</code> used)	<code>Grid[1][2]</code> fails parsing; use <code>(Grid[1])[2]</code>
Rank Polymorphism	Supported automatically	Not supported (opaque nested descriptors)
Cache & Vectorization	Optimal L1/L2 locality, SIMD/BLAS	Pointer indirection, no vectorization

Table 5.1: Comparison of Flat vs. Nested `array_dv` Representations.

Common Syntax Confusion: Chained Subscripting `Grid[1][2]`

In standard Sisal grammar, writing `Grid[1][2]` directly on a single line produces a syntax parser error because `[2]` is not recognized as a trailing suffix on `Grid[1]`. To index into a nested jagged array, developers must use intermediate let-bindings (`R1 := Grid[1]; Elem := R1[2]`) or parenthesized subscripting `(Grid[1])[2]`.

Performance Guidance: Prefer Flat `array_dv[T]`

While nested `array_dv[array_dv[T]]` structures compile and execute cleanly for irregular or jagged data collections, they introduce pointer indirection and forfeit rank polymorphism, zero-copy slicing, and SIMD vectorization. Developers should use flat `array_dv[T]` for all dense linear algebra and tensor computations, reserving nested arrays strictly for non-uniform data structures.

5.5 Sisal-2026 Rank-Polymorphic Examples

5.5.1 1. Matrix-Vector & Matrix-Scalar Operations

In Sisal-2026, rank polymorphism operates on `array_dv[T]` arguments along with scalar values. Built-in binary operators (`+`, `-`, `*`, `/`) and functions accepting `array_dv` arguments inspect the dynamic rank metadata of their inputs and lift automatically across higher-dimensional dope vectors:

```

1 function TransformAndScale( M : array_dv[real]; V : array_dv[real]; s
  : real returns array_dv[real] )
2   let
3     % M is a 2D Matrix of Shape [2, 3]
4     % V is a 1D Vector of Shape [3]
5     % s is a scalar real (0D)
6
7     % 1. Matrix + Vector broadcast addition -> returns Shape [2, 3]
```

```

8      M_plus_V := M + V;
9
10     % 2. Matrix-Scalar broadcast multiplication -> returns Shape [2, 3]
11     Scaled := M_plus_V * s;
12
13     % 3. Combined inline rank-polymorphic expression
14     Result := (M * s) + V
15 in
16     Result
17 end let
18 end function

```

Listing 5.3: Rank-Polymorphic Matrix-Vector & Scalar Expressions

Rank-Polymorphic Operand Scope

Rank-polymorphic lifting applies directly to built-in operators and functions defined with `array_dv[T]` parameters (along with scalar values). To apply a scalar-only function `f(x, y : real)` over array elements, use the whole-array `MAP(A, B, f)` primitive or define the function signature with `array_dv[real]` parameters.

5.6 Array Construction: Flat 1D Literals & reshape

Array literal syntax `array_dv [1: v1, v2, ...]` constructs a **1D flat vector**. To construct a multi-dimensional matrix or tensor from literal values in Sisal-2026, developers initialize a flat 1D vector literal and reshape it using the built-in `reshape(A, dim0, dim1, ...)` function, or construct it dynamically via parallel `forall` loop gather reductions.

```

1 function MatrixInit( returns array_dv[real] )
2   let
3     % 1. Construct a flat 1D vector literal (16 elements)
4     Flat := array_dv [1: 1.0, 2.0, 3.0, 4.0,
5                          5.0, 6.0, 7.0, 8.0,
6                          9.0, 10.0, 11.0, 12.0,
7                          13.0, 14.0, 15.0, 16.0];
8
9     % 2. Reshape into a 4x4 rank-2 matrix (O(1) dope vector view shift)
10    M := reshape(Flat, 4, 4)
11 in
12    M
13 end let
14 end function

```

Listing 5.4: Multi-Dimensional Array Construction via reshape

5.6.1 2. Zero-Copy Slicing & Explicit Range Notation

Slicing a matrix creates a zero-copy metadata view with $O(1)$ constant time complexity:

```

1 function SlicingDemo( returns array_dv[real] )
2   let
3     % Construct 4x4 Matrix by reshaping flat 1D literal
4     Flat := array_dv [1: 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0,
5                          9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0,
6                          16.0];
7
8     M := reshape(Flat, 4, 4);

```

```

7      % Sub-matrix view (Rows 2..3, Cols 1..2) -> Shape [2, 2]
8      SubView := M[2..3, 1..2];
9
10
11     % Explicit row slice using range notation: Extract row 2 -> 1D
12     % Vector of Shape [4]
13     RowVector := M[2, 1..4]
14
15     in
16     SubView + RowVector % Automatic broadcasting!
17 end let
18 end function

```

Listing 5.5: Zero-Copy Slicing in Sisal-2026

5.7 Indexing Ambiguity & Explicit Slicing Notation ($A[i]$ vs. $A[i, \dots]$)

A fundamental design decision in Sisal-2026 is disambiguating single-subscript indexing vs. slicing.

5.7.1 The Lowering Problem with $A[i]$

In legacy ragged array languages (such as Sisal 1.2 or C nested arrays), a 2D matrix is an array of pointers (`array[array[T]]`). In that pointer model, single-index access $A[i]$ dereferences the outer pointer to return a 1D row array `array[T]`.

However, in Sisal-2026, all dense arrays use flat dope vectors (`array_dv[T]`). If single-subscript indexing $A[i]$ were allowed to return a 1D row vector when A is a 2D matrix, but a scalar T when A is a 1D vector, the IF1 IR lowering and static type checking of $A[i]$ would be ambiguous:

- Does $A[i]$ lower to a scalar element access (`DOPE_VECTOR_ELEMENT`) or a multi-element slice view (`DOPE_VECTOR_SLICE`)?
- What is the static return type of $A[i]$ in the compiler type checker?

5.7.2 The Sisal-2026 Solution

Sisal-2026 enforces strict, deterministic syntax rules:

1. **Single Subscript Indexing ($A[i]$): ALWAYS returns a scalar value T !**
 - For a 1D vector V , $V[i]$ returns the i -th scalar element.
 - For a 2D matrix M , single-subscript $M[i]$ performs **linear (flat 1D) indexing** into the underlying row-major memory buffer as if M were a single flattened 1D row! Thus, $M[i]$ extracts the i -th scalar element of the flat matrix.
 - 2D coordinate scalar access uses two-subscript indexing $M[i, j]$.
2. **Explicit Row/Column Slicing ($A[i, \dots]$ or $A[i, 1..N]$):** To explicitly extract a 1D row vector slice from a 2D matrix, developers use range slicing notation $A[i, \dots]$ or $A[i, 1..N]$.

```

1 function IndexVsSliceDemo( M : array_dv[real] returns real,
   array_dv[real] )
2 let
3   % 1. Coordinate Indexing: M[2, 3] ALWAYS returns a scalar real
   element
4   scalar_val := M[2, 3];
5
6   % 2. Explicit Row Slicing: M[2, ..] explicitly extracts Row 2 as a
   1D Vector view
7   row_slice := M[2, ..];
8
9   % 3. Explicit Column Slicing: M[.., 3] explicitly extracts Column
   3 as a 1D Vector view
10  col_slice := M[.., 3]
11 in
12   scalar_val, row_slice
13 end let
14 end function

```

Listing 5.6: Indexing vs. Explicit Slicing in Sisal-2026

Why This Is a Major Architectural Advantage

Enforcing that $M[i, j]$ is scalar element access while $M[i, ..]$ is slice view extraction eliminates type ambiguity in the compiler frontend. The type checker statically knows whether an expression produces a scalar T or a metadata dope vector view $\text{array_dv}[T]$, leading to clean IF1 node lowering and zero runtime type-check overhead.

5.8 Broadcasting Rules: Right-Aligned Trailing Axis Agreement

Sisal-2026 enforces right-aligned trailing axis broadcasting rules (matching NumPy and PyTorch conventions):

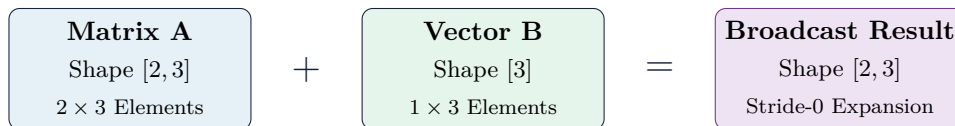


Figure 5.2: Right-Aligned Stride-0 Broadcasting Alignment.

When evaluating $A \oplus B$:

1. **Right Alignment:** Align dimension shapes starting from the rightmost (trailing) axis.
2. **Axis Compatibility:** For each aligned axis d :
 - If $\text{shape_A}[d] == \text{shape_B}[d]$, the dimensions agree.
 - If one dimension size is 1, it is expanded to match the other size by setting its byte stride to 0.
 - If dimension sizes differ and neither is 1, a runtime shape error is raised.
3. **Zero-Copy Stride-0 Expansion:** Axis expansion incurs 0 byte copies because setting $\text{stride} = 0$ reuses existing element memory repeatedly without duplication.

5.9 IF1 Compiler Lowering & Execution Pass

During AST lowering to the IF1 dataflow graph:

1. Binary arithmetic operations between `array_dv` operands lower to `CONFORM_CHECK` nodes.
2. `CONFORM_CHECK` verifies shape compatibility and calculates output shape and stride-0 metadata.
3. Elementwise computation loops iterate over total elements using `sisal_dv_offset_at` to compute flat memory addresses.

```

1 inline int32_t sisal_dv_offset_at(const sisal_array_t* arr, int32_t
  flat_idx) {
2     int32_t offset = arr->offset;
3     int32_t rem = flat_idx;
4     for (int32_t i = 0; i < arr->rank; ++i) {
5         int32_t dim_size = arr->shape[i];
6         int32_t coord = rem / arr->stride_mod[i];
7         rem %= arr->stride_mod[i];
8         % Stride of 0 automatically zeroes out offset addition for
          broadcast axes!
9         offset += coord * arr->stride[i];
10    }
11    return offset;
12 }
```

Listing 5.7: C++23 Runtime Stride-0 Index Calculation

5.10 Summary of Benefits in Sisal-2026

Feature	Legacy Sisal 1.2 / 2.0	Sisal-2026 (<code>array_dv</code>)
Memory Layout	Ragged nested pointers (<code>array[array[T]]</code>)	Contiguous flat buffer with $O(1)$ dope vector
Slicing Cost	$O(N)$ allocation & copying	$O(1)$ constant-time metadata view shift
Broadcasting	Manual nested <code>forall</code> loops	Automatic right-aligned zero-copy stride-0
Single Assignment	Static update-in-place pass	Native Copy-on-Write reference counting
Hardware Acceleration	Manual vectorization	Direct BLAS/LAPACK (<code>cblas_dgemm</code>)

Table 5.2: Comparison of Array Models.

Chapter 6

Expressions, Operators & SIMD Intrinsics

6.1 Arithmetic, Relational & Logical Expressions

Sisal-2026 supports standard mathematical operators with traditional precedence rules:

Category	Operators	Description
Arithmetic	+, -, *, /, **, mod	Addition, Subtraction, Multiplication, Division, Power, Modulo
Relational	=, ~=, <, <=, >, >=	Equal, Not Equal, Less, Less-Equal, Greater, Greater-Equal
Logical	and, or, not	Boolean operations
Bitwise	bit_and, bit_or, bit_xor	Low-level bitwise operations

Table 6.1: Operator Precedence & Categories.

6.2 Pattern Matching & Wildcard Bindings

Sisal-2026 introduces pattern matching with wildcard (_) support in `let` bindings, assignment expressions, and function returns:

```
1 function TupleUnpack( returns integer, real, boolean )
2   10, 3.14, true
3 end function
4
5 function Main( returns integer )
6   let
7     % Ignore second return value using wildcard _
8     val, _, flag := TupleUnpack();
9     _ := printf("Flag value: %d\n", if flag then 1 else 0 end if)
10  in
11    val
12  end let
13 end function
```

Listing 6.1: Wildcard Pattern Matching Examples

6.3 First-Class Higher-Order Functions (HOFs) & Closures

Functions in Sisal-2026 are first-class values. Lambda expressions create functional closures that capture environment variables:

```

1 function ApplyTwice( f : function(integer returns integer); x :
  integer returns integer )
2   f(f(x))
3 end function
4
5 function TestHOF( returns integer )
6   let
7     multiplier := 5;
8     % Lambda closure capturing multiplier
9     scale := function( v : integer returns integer ) v * multiplier
10    end function;
11    res := ApplyTwice(scale, 2) % Computes (2 * 5) * 5 = 50
12  in
13    res
14 end let
15 end function

```

Listing 6.2: Higher-Order Function & Closure Usage

6.4 Hardware SIMD Intrinsics (float4, mat2)

To exploit CPU vector execution units (ARM Neon, x86 AVX-512) and GPU shader cores, Sisal-2026 introduces fixed-size SIMD vector and matrix types:

SIMD Type	Components	Target Alignment
float2, int2	2 single-precision floats / ints	64-bit SIMD lane
float3	3 single-precision floats	96-bit SIMD lane
float4, int4	4 single-precision floats / ints	128-bit SIMD lane
mat2	2 × 2 matrix	128-bit SIMD matrix register
mat3	3 × 3 matrix	288-bit SIMD register block
mat4	4 × 4 matrix	512-bit SIMD register block

Table 6.2: Fixed SIMD Vector & Matrix Primitives.

```

1 function TransformVector( v : float4; m : mat4 returns float4 )
2   let
3     % Direct SIMD matrix-vector multiplication compiled to single
4     AVX/Neon instruction
5     v_out := m * v;
6     norm  := dot(v_out, v_out)
7   in
8     v_out
9   end let
10 end function

```

Listing 6.3: SIMD Math Intrinsics Example

Chapter 7

Einstein Summation & Bulk Array Operations

7.1 Einstein Summation Engine (EINSUM)

Sisal-2026 includes a built-in Einstein summation contraction engine. The syntax parses standard multi-tensor subscript notation:

```
1 C := EINSUM("ij,jk->ik", MatrixA, MatrixB)
```

Listing 7.1: Einstein Summation Syntax

7.1.1 Lowering Fast-Paths

The compiler inspects EINSUM subscript strings during AST lowering (`einsum_lower.ml`) and routes contractions to hardware BLAS fast-paths:

- "ij,jk->ik": Matrix-Matrix Multiplication $\rightarrow$ `cblas_dgemm` / `cblas_sgemm`.
- "ij,j->i": Matrix-Vector Multiplication $\rightarrow$ `cblas_dgemv` / `cblas_sgemv`.
- "i,i->": Vector Dot Product $\rightarrow$ `cblas_ddot` / `cblas_sdot`.
- "ij->ji": Matrix Transposition $\rightarrow$ $O(1)$ stride swap in dope vector metadata.

For general multi-index contractions not matching single 2D BLAS signatures, the compiler lowers the expression to an IF1 EINSUM_NODE multi-loop iteration graph.

7.1.2 High-Rank Tensors & Contraction Path Optimization

While MATMUL and INNERPRODUCT provide direct, concise syntax for standard 2D matrix operations, EINSUM serves two critical structural roles in tensor compilation:

1. **High-Rank Tensor Contractions:** For 3D, 4D, and 5D attention graphs (e.g. $Q, K, V \in \mathbb{R}^{B \times H \times S \times D}$ in Grouped Query Attention), `EINSUM("bhqd,bhkd->bhqk", Q, K)` declaratively specifies contraction axes without manual reshape or permutation boilerplate.
2. **Multi-Contraction Chain Optimization (Compiler Roadmap Target):** When chaining multiple matrix/tensor products in sequence (e.g., $P \cdot V \cdot W_O$), exposing the contraction topology through EINSUM provides the structural foundation for **Dynamic Programming Contraction Path Optimization** (on the compiler optimizer roadmap). This pass will automatically evaluate intermediate tensor dimension products to select the optimal parenthesization order that minimizes total FLOP complexity (reducing $O(N^4)$ naive contractions down to optimal $O(N^3)$ sequences) before emitting BLAS kernel calls.

7.1.3 Tiled EINSUM Subscripts & Unified Permutation (Wishlist Item)

An upcoming design proposal on the Sisal-2026 compiler wishlist extends the EINSUM subscript string to natively unify **tensor contraction math**, **L1/L2 cache tiling**, and **loop permutation** into a single bracketed notation, eliminating separate imperative transformation scripts.

In this syntax, bracket nesting depth represents tile/cache depth (e.g. (i 32) represents a 32-element L2 block tile), while the left-to-right order of terms on the output side (->) dictates the execution loop nesting:

```

1 % Tiled Matrix Multiplication (32x64x16) with k as Middle Loop
2 C := EINSUM("((i 32) (j 64)) ((j 64) (k 16)) -> (i 32) (k 16) (j 64)",
  A, B)

```

Listing 7.2: Tiled EINSUM Subscript with Middle k Loop

Step-by-Step Lowering Walkthrough (Pseudo-Fortran)

To understand why placing k as the middle loop delivers superior performance, consider the step-by-step compiler lowerings from naive untiled execution down to tiled k -in-the-middle micro-kernels:

1. Naive Untiled Lowering ("ik, kj -> ij" $\rightarrow i \rightarrow j \rightarrow k$ order):

```

1 ! Naive Lowering: Reduction k in innermost loop (Strided access on
  B)
2 DO i = 1, M
3   DO j = 1, N
4     sum_val = 0.0d0
5     DO k = 1, K
6       sum_val = sum_val + A(i, k) * B(k, j)    ! B(k, j) jumps N
          elements every step
7     END DO
8     C(i, j) = sum_val
9   END DO
10 END DO

```

2. Interchanged Lowering (k as Middle Loop $\rightarrow i \rightarrow k \rightarrow j$ order):

```

1 ! Interchanged: k in middle position (Scalar broadcast of A,
  unit-stride for B and C)
2 DO i = 1, M
3   DO k = 1, K                                ! k is MIDDLE loop!
4     a_val = A(i, k)                          ! Broadcast: Load once into
          register
5     DO j = 1, N                                ! Innermost loop over j
          (Vector unit-stride)
6       C(i, j) = C(i, j) + a_val * B(k, j)
7     END DO
8   END DO
9 END DO

```

3. Tiled Lowering with k in Middle ("((i 32) (j 64)) ((j 64) (k 16)) -> (i 32) (k 16) (j 64)":

```

1 ! Tiled & Interchanged: i_tile -> k_tile -> j_tile -> i_elem ->
  k_elem -> j_elem

```

```

2 DO i_tile = 1, M, 32
3   DO k_tile = 1, K, 16                                ! k_tile is MIDDLE tile
4     DO j_tile = 1, N, 64
5       ! Inner Micro-kernel (L1 Cache / SIMD Vector Registers)
6       DO i_elem = i_tile, MIN(i_tile + 31, M)
7         DO k_elem = k_tile, MIN(k_tile + 15, K)      ! k_elem is
8           MIDDLE element loop!
9           a_val = A(i_elem, k_elem)
10
11           !DIR$ SIMD (Vectorized unit-stride loop over j_elem)
12           DO j_elem = j_tile, MIN(j_tile + 63, N)
13             C(i_elem, j_elem) = C(i_elem, j_elem) + a_val *
14               B(k_elem, j_elem)
15           END DO
16         END DO
17       END DO
18     END DO
19   END DO
20 END DO

```

Related Work & Novelty Analysis

A comparison of the Sisal-2026 tiled EINSUM proposal with state-of-the-art tensor compilers and programming frameworks highlights its unique design positioning:

- **NumPy / PyTorch einsum:** Uses standard Einstein notation ("ik,kj->ij"). Expresses contraction math declaratively, but has **zero syntax for loop tiling, cache block sizes, or execution loop ordering**.
- **Halide (Ragan-Kelley et al., PLDI 2013) & TVM (Chen et al., OSDI 2018):** Pioneered the decoupling of computation algorithms from execution schedules. However, scheduling is imperative and separate from the math expression (e.g. `s.tile(i, i_out, i_in, 32).reorder(i_out, k_out, j_out)` in an external Python script).
- **Einops (Rozenberg, ICLR 2022):** Provides index rearrangement syntax ("b (h w) -> b h w"), but focuses exclusively on spatial array reshaping and tensor broadcasting—**cannot specify contraction reductions, cache block sizes, or SIMD loop order**.
- **Tensor Comprehensions (Vasilache et al., FAIR 2018) & TACO (Kjolstad et al., OOPSLA 2017):** Uses polyhedral compiler infrastructure (ISL) to discover tile sizes automatically during compilation. No inline syntax exists for the developer to specify tile bounds directly in the string.
- **Sisal-2026 Novelty:**
 1. **Purely Functional Inline Scheduling:** Unifies math definition and hardware loop schedule in a single functional string expression.
 2. **Zero-Keyward Positional Ordering:** The position of bracketed terms on the RHS of -> dictates outer/inner loop nesting and loop interchange (e.g. -> (i 32) (k 16) (j 64) places *k* in the middle).
 3. **Cache Depth Mapping:** Bracket depth corresponds 1-to-1 to physical memory levels: *i* (flat DRAM), (*i* 32) (L2 Cache), ((*i* 128) 8) (L1 / Vector Register).

Work-in-Progress Disclaimer

This documentation and manual are works in progress. Because compiler optimization passes, AST lowerings, and backend code generators are actively evolving, developers should examine the execution logs, generated C++ files, and benchmark results in `test/e2e/` to inspect the authoritative current state of the compiler's optimization capabilities.

7.2 APL-Style Bulk Operations Battery (58 Primitives)

Sisal-2026 provides 58 whole-array bulk primitives, eliminating manual parallel `forall` boilerplate code for standard array manipulations:

Category	Primitive Names	Semantics & Description
Reductions	SUM, PRODUCT, LEAST, GREATEST, MEAN, VARIANCE, STDDEV, NORM	Reduce array along default or specified axis into scalar value.
Scans	SCAN, CUMSUM, CUMPROD	Prefix scan operation along leading axis.
Structural	TAKE, DROP, ROTATE, REVERSE, COMPRESS, RAVEL, EXPAND, SQUEEZE, PERMUTE	Structural reshaping and axis permuting operations.
Selection	SORT, ARGMAX, ARGMIN, GRADE_UP, GRADE_DOWN, NONZERO, WHERE	Sorting and index mask extraction.
Higher-Order	MAP, FOLDL, FOLDR, STENCIL, INNERPRODUCT, OUTERPRODUCT	Functional combinators across multidimensional dope vectors.

Table 7.1: APL Bulk Operation Battery Summary.

```

1  let
2    A := array_dv [1: 5.0, 2.0, 9.0, 1.0, 7.0];
3
4    % Calculate Mean and Standard Deviation
5    m := MEAN(A);
6    s := STDDEV(A);
7
8    % Sort and Grade Indices
9    SortedA := SORT(A);           % [1.0, 2.0, 5.0, 7.0, 9.0]
10   MaxIdx  := ARGMAX(A);          % 3 (index of value 9.0)
11
12   % Stencil Convolution
13   Smooth := STENCIL(A, 3, function(window : array_dv[real] returns
14     real)
15     (window[1] + window[2] + window[3]) / 3.0
16   end function)
17 in
18   SortedA, MaxIdx, Smooth
end let

```

Listing 7.3: Bulk Operation Battery Code Examples

Chapter 8

Parallel & Sequential Loops

8.1 Parallel Loops (for)

The parallel for loop (lowered to `ForallNode` compound graphs in IF1 IR) is the primary construct for data-parallel execution in Sisal-2026. Every iteration of a parallel `for` loop is independent and can execute concurrently on separate CPU worker threads or GPU thread blocks.

Keyword Clarification: `for` vs. `forall`

In Sisal-2026 source code, the keyword for parallel loops is `for` (e.g., `for i in 1..N ... end for`). The term *forall* refers to the underlying IF1 dataflow compound graph node (`ForallNode`) and literature terminology.

8.1.1 Syntax & Generator Ranges

A parallel for loop consists of three sections:

1. **Generator Range:** Defines the index domain (`i in 1, N` or element iteration `x in Array`).
2. **Loop Body:** Computes expression bindings for each element in the domain.
3. **Returns Clause:** Specifies reduction accumulators (`sum of`, `array of`, `stream of`, `product of`, `least of`, `greatest of`).

```
1 function VectorScale( A : array_dv[real]; factor : real returns
   array_dv[real] )
2   for x in A
3     y := x * factor
4   returns
5     array of y
6   end for
7 end function
```

Listing 8.1: Parallel for Loop Example

8.1.2 Cross Products & Dot Products

Multiple generator ranges can be combined using `cross` (Cartesian product) or `dot` (lock-step parallel iteration):

```
1 % Cross Product (N x M iterations)
2 Matrix := for i in 1, N cross j in 1, M
3   val := i * 10 + j
4 returns
5   array of val
```

```

6 end for;
7
8 % Dot Product (min(N, M) lock-step iterations)
9 Sums := for x in VectorA dot y in VectorB
10   z := x + y
11 returns
12   array of z
13 end for;

```

Listing 8.2: Cross Product vs. Dot Product Generators

8.2 Sequential Loops (for initial)

When loop iterations have loop-carried dependencies (where iteration k requires results from iteration $k - 1$), the `for initial` sequential loop is used.

```

1 function Fibonacci( n : integer returns integer )
2   for initial
3     i := 1;
4     a := 0;
5     b := 1
6   while i < n repeat
7     i := old i + 1;
8     a := old b;
9     b := old a + old b
10  returns
11    value of b
12  end for
13 end function

```

Listing 8.3: Sequential Loop with Zero-Copy Tuple Swapping

old Keyword

Inside a `for initial` loop body, the `old` keyword references value bindings from the previous iteration. The compiler lowers state updates to zero-copy pointer swaps.

8.3 Loop Semantics & Comparative Analysis: Sisal vs. C vs. Fortran

Sisal's `for initial` construct is a functional state-transition graph loop rather than a control-flow jump. This fundamental semantic difference distinguishes Sisal loops from traditional imperative loops in C and Fortran.

8.3.1 Core Rules of for initial Iteration

1. **Rule 1: Unconditional Initial Seed Evaluation (No Undefined States):** Sisal defines a single-assignment value history that begins with an explicit initial state (`body_0`). The `initial` seed is unconditionally evaluated and added to any accumulation collection (e.g. `returns stream of I`) *before* the loop condition check occurs. Even in a zero-trip loop where the condition is false on entry, the initial seed value is safely defined and gathered, eliminating any risk of reading garbage or uninitialized memory.

2. **Rule 2: Out-of-Bounds Value Calculation & Gathering:** In Sisal, the loop body computes the new state of loop variables from the previous state ($I := \text{old } I + \text{step}$). If the condition is satisfied, the body computes the new state and immediately gathers it. Consequently, the final value of I (which fails the subsequent condition check and terminates the loop) has already been computed and gathered into the result collection.

8.3.2 Comparative Analysis: Zero-Trip & Normal Loops

Table 8.1 and Table 8.2 compare the execution behavior of Sisal, C/C++, and Fortran under zero-trip and normal loop scenarios.

Language	Example Syntax	Execution Details & Undefined Risk
Sisal-2026	for initial I := 10 while I < 5 repeat I := old I + 1 returns stream of I end for	Initial seed executes unconditionally and gathers 10. Condition $10 < 5$ is false $\rightarrow$ exits. Stream: [10] (Size 1) Risk: None.
C / C++	int r; for(int i=10; i<5; i++) r=i;	Condition $10 < 5$ is checked first and fails. Body never executes. Vector: [] (Empty) Risk: High (reading r yields garbage/UB).
Fortran	INTEGER :: I, R; DO I = 10, 5; R = I; END DO	Trip count $\text{MAX}(0, 5-10+1)$ evaluates to 0. Body never executes. Result: Uninitialized Risk: High.

Table 8.1: Zero-Trip Loop Behavior Comparison (Initial Seed = 10, Bound = 5).

Language	Example Syntax	State Sequence & Resulting Collection
Sisal-2026	for initial I := 1 while I <= 5 repeat I := old I + 1 returns stream of I end for	1. Seed gathers 1. 2. Iterations 1–5 compute and gather 2, 3, 4, 5, 6. 3. $6 \leq 5$ fails $\rightarrow$ exits. Stream: [1, 2, 3, 4, 5, 6] (Size 6)
C / C++	for(int i=1; i<=5; i++) vec.push_back(i);	1. Iterations 1–5 push 1, 2, 3, 4, 5. 2. Increment to 6; $6 \leq 5$ fails $\rightarrow$ exits. Vector: [1, 2, 3, 4, 5] (Size 5)
Fortran	DO I = 1, 5; ... END DO	Loop executes 5 times. Final I is incremented to 6 on exit.

Table 8.2: Normal 5-Iteration Loop Behavior Comparison (Range = 1..5).

8.3.3 Key Design Takeaways

- **Collection Sizes:** For N successful condition evaluations, a traditional C/Fortran loop produces N elements, whereas a Sisal `for initial` loop yields $N + 1$ elements (the initial seed plus N body calculations).
- **Initialization Safety:** Sisal guarantees that loop-carried value streams always contain at least the initial seed value, statically eliminating uninitialized memory access.

Chapter 9

Streams & Stackless Coroutines

9.1 Stream Abstraction (`stream[T]`)

A `stream` in Sisal-2026 represents a potentially infinite sequence of lazily evaluated values. Unlike dense arrays (`array_dv`), streams do not require materializing all elements in memory concurrently.

9.2 C++20 Stackless Coroutines (`co_yield`)

In the Sisal-2026 compiler, stream generators compile directly into C++20 stackless coroutines (`sisal_generator<T>`).

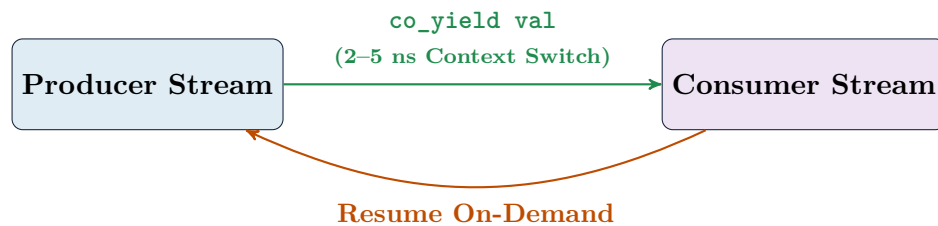


Figure 9.1: Zero-Allocation Coroutine Stream Switching.

```
1 function GenerateIntegers( start, limit : integer returns
   stream[integer] )
2   for initial
3     val := start
4     while val <= limit repeat
5       val := old val + 1
6     returns
7       stream of val
8   end for
9 end function
```

Listing 9.1: Stream Generator Function

9.3 Heap Allocation Elision (HALO) Performance

Because C++20 coroutine frames fit within standard frame boundaries when inlined, modern compilers (GCC 13+, Clang 16+) perform Heap Allocation Elision Optimization (HALO). This places coroutine frame state directly on the caller stack frame, achieving:

- **Context Switch Overhead:** 2 to 5 nanoseconds ($30\times$ – $100\times$ faster than POSIX `ucontext` or OS thread context switches).
- **Memory Overhead:** Zero heap allocation overhead for stream pipelines.

Chapter 10

Side-Effect Monad Ordering

10.1 Reconciling Pure Functional Dataflow with I/O

Pure dataflow languages traditionally struggle with imperative I/O operations. In a pure dataflow graph (such as Sisal 1.2), node execution is driven strictly by data dependencies: a node executes as soon as all of its input arguments become available. When sub-expressions have no data dependencies between them, they are scheduled concurrently across available processor threads or executed in arbitrary order.

While this non-deterministic out-of-order execution model is ideal for high-performance data parallelism, it presents severe challenges for practical programming tasks:

- **Console Output Interleaving:** Concurrent printing from multiple parallel sub-graph branches causes character streams to collide and interleave unpredictably.
- **Debug Tracing:** Programmers cannot easily trace algorithm iteration steps or log intermediate values when execution timing varies between runs.
- **Sequential File/Stream Access:** Operations like writing consecutive record chunks to stdout or disk files require a guaranteed, deterministic order.

Sisal-2026 solves this challenge by introducing **Monad Control Ports** into the compiler's IF1 intermediate representation graph.

10.2 What is a Monad in Sisal-2026?

In functional programming and category theory, a *monad* is a design pattern that structures computations by encapsulating value evaluation alongside side-effect execution contexts. In Sisal-2026, a monad represents an implicit, linearly-threaded sequencing state that enforces a strict execution order between side-effecting operations without violating the language's core single-assignment immutability rules.

Conceptually, a side-effecting built-in function in Sisal-2026 operates as a state-transformer monad:

$$f :: A \rightarrow S \rightarrow (B, S) \tag{10.1}$$

where A is the input argument, B is the returned result value, and S represents an abstract "World State" token. Rather than exposing state tokens to the programmer, the compiler automatically threads this state token through side-effecting operations, creating an explicit linear execution sequence.

10.3 Serialization & Imperative Style for Practical Scenarios

To handle practical software engineering requirements—such as progress logging, error reporting, and stream output—Sisal-2026 enables developers to write familiar, imperative-style sequential code within pure functional routines.

When a developer writes consecutive side-effecting statements:

```

1 function ExecutePipeline( data : array_dv[double] returns
   array_dv[double] )
2   let
3     _ := printf("Phase 1: Starting computation...\n");
4     R1 := ProcessPhase1(data);
5     _ := printf("Phase 2: Intermediate processing complete.\n");
6     R2 := ProcessPhase2(R1);
7     _ := printf("Phase 3: Finalizing results.\n")
8   in
9     R2
10  end let
11 end function

```

Listing 10.1: Imperative-Style Logging Sequence in Sisal-2026

Sisal-2026 reconciles this imperative notation with its dataflow engine through **deterministic serialization**:

1. **Localized Imperative Sequencing:** Side-effecting statements are executed in exact lexicographic (source file) order.
2. **Concurrent Functional Subgraphs:** Pure mathematical expressions (`ProcessPhase1`, `ProcessPhase2`, array arithmetic) continue to execute with maximum data parallelism across all available CPU worker threads.
3. **No Non-Deterministic Race Conditions:** Output buffers are never corrupted by out-of-order thread scheduling.

10.4 Monad Control Ports in the IF1 Intermediate Representation

A **Monad Control Port** is a compiler-inserted IR graph port designed to carry sequence control tokens. When the Sisal-2026 parser encounters side-effecting built-in procedures (`printf`, `cout`, `cerr`), it decorates the corresponding IF1 AST nodes with dedicated monad ports:

- `PRINTF_TY`: Monad sequence control port for standard output logging.
- `COUT_TY`: C++ stream output monad sequence port.
- `CERR_TY`: Standard error stream monad sequence port.

During graph lowering, the compiler constructs artificial control-dependency edges (*Monad Control Edges*) between consecutive side-effecting nodes based on their program text position. An IF1 node possessing an incoming Monad Control Edge is prohibited from firing until the predecessor side-effecting node has completed execution and emitted its control token.

As illustrated in Figure 10.1, pure computation paths (`HeavyCompute()`) execute asynchronously according to data availability, while side-effecting nodes are linked via the dashed `PRINTF_TY` Monad Control Edge to guarantee strict ordering.

10.5 Value Pass-Through vs. Wildcard Idioms

Sisal-2026 supports two primary statement idioms for `printf` logging:

- **Wildcard Binding** (`_ := printf(...)`): Evaluates the formatted logging statement and discards the return value using the wildcard pattern `_`. This pattern is ideal for stand-alone progress messages.

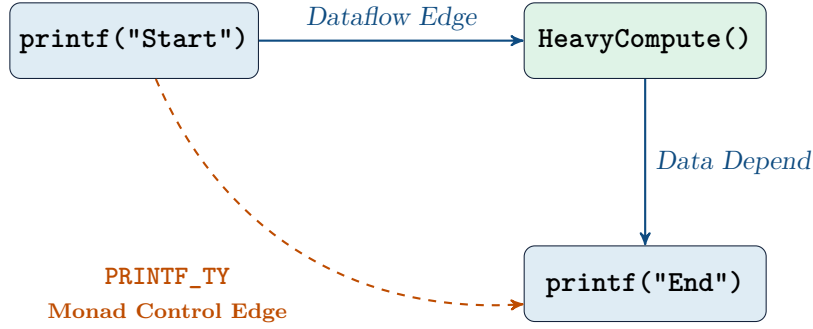


Figure 10.1: Monad Control Port Dependency Edge in IF1 Graph.

- **Value Pass-Through** (`res := printf("%d\n", val)`): Logs the formatted message while transparently evaluating and returning the printed value argument. This permits inline debug logging without disrupting dataflow or polluting the program with temporary variable declarations.

10.6 Deterministic Lexicographic Ordering Guarantees

By combining Monad Control Ports in the IR with single-assignment language semantics, Sisal-2026 achieves a powerful synthesis:

1. **Guaranteed Determinism:** Given identical inputs, a Sisal-2026 program will produce the exact same console log output in the exact same sequence across every run, regardless of hardware core count or thread scheduling order.
2. **Zero Impact on Pure Parallelism:** Monad control edges sequence only nodes decorated with side-effect ports. Independent mathematical operations, array contractions, tensor operations, and loop iterations remain 100% parallelizable across multicore targets.

Chapter 11

The IF1 Intermediate Representation

11.1 IF1 Dataflow Graph Architecture

The core intermediate representation of the Sisal-2026 compiler is **IF1** (*Intermediate Format 1*), an acyclic directed dataflow graph format.

An IF1 graph consists of:

- **Nodes:** Representing operations, function calls, or subgraphs.
- **Edges:** Representing data values flowing from output ports of source nodes to input ports of destination nodes.
- **Types:** Attached to every edge to enforce type safety.

11.2 Simple vs. Compound Nodes

IF1 distinguishes between two primary categories of nodes:

1. **Simple Nodes:** Primitive scalar arithmetic, dope vector indexing, logic, or function calls (e.g. `Plus`, `Times`, `DopeVectorSlice`).
2. **Compound Nodes:** Complex control structures containing nested subgraphs (e.g. `SelectNode` for conditionals, `ForallNode` for parallel loops, `LoopANode`/`LoopBNode` for sequential loops).

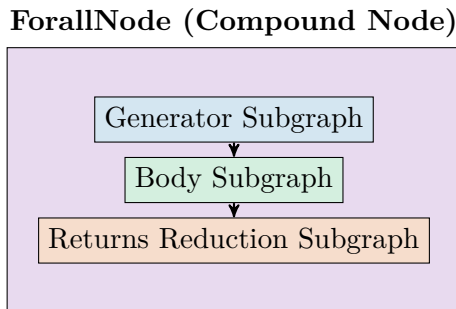


Figure 11.1: Structure of an IF1 Compound `ForallNode`.

11.3 Optimization Passes

The Sisal-2026 compiler executes several optimization passes directly on IF1 graphs:

- **Common Subexpression Elimination (CSE):** Merges identical pure subgraphs.
- **Dead Code Elimination (DCE):** Removes unreferenced node outputs.

- **Loop Invariant Code Motion (LICM):** Hoists constant expressions out of sequential loops.
- **Copy Elimination & Static Liveness:** Eliminates array buffer copies whenever reference counts equal 1.

Chapter 12

C++23 Runtime Architecture

12.1 `sisal_runtime.h` Execution Engine

The Sisal-2026 code generator targets modern C++23. The runtime layer is implemented in `runtime/sisal_runtime.h` and provides header-only execution primitives for dope vectors, parallel loops, SIMD intrinsics, and BLAS routines.

12.2 Memory Management & Reference Counting

Memory allocation for `array_dv` elements is managed via aligned heap allocators. Reference counting operates on the dope vector metadata:

```
1 inline sisal_array_t* sisal_dv_retain(sisal_array_t* arr) {
2     if (arr) {
3         arr->ref_count++;
4     }
5     return arr;
6 }
7
8 inline void sisal_dv_release(sisal_array_t* arr) {
9     if (arr && --(arr->ref_count) == 0) {
10        aligned_free(arr->data);
11        free(arr->shape);
12        free(arr->stride);
13        free(arr);
14    }
15 }
```

Listing 12.1: Dope Vector Reference Counting API

12.3 Hardware BLAS & LAPACK Integration

For matrix-matrix and matrix-vector operations generated by EINSUM or bulk primitives, the runtime interfaces with hardware-accelerated BLAS libraries (Apple Accelerate framework on macOS, OpenBLAS / MKL on Linux):

```
1 inline sisal_array_t* sisal_dv_matmul(sisal_array_t* A, sisal_array_t*
   B) {
2     % Verify rank-2 compatibility and stride alignment
3     int M = A->shape[0];
4     int K = A->shape[1];
5     int N = B->shape[1];
6
7     sisal_array_t* C = sisal_dv_alloc_2d(M, N, sizeof(double));
8
9     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
10        M, N, K, 1.0,
```

```
11         (const double*)A->data, A->stride[0],  
12         (const double*)B->data, B->stride[0],  
13         0.0, (double*)C->data, C->stride[0]);  
14  
15     return C;  
16 }
```

Listing 12.2: BLAS DGEMM Invocation in C++23 Runtime

Chapter 13

Copy Elimination, Liveness Analysis & Memory Reuse

13.1 The Single-Assignment Memory Challenge

A foundational principle of pure functional dataflow languages is *single assignment*: once an array variable is bound, its contents are mathematically immutable. However, naive enforcement of immutability would require generating a full $O(N)$ deep memory copy (`memcpy`) whenever an array element or slice is modified (e.g., $A' = A[i \leftarrow v]$). In large-scale scientific computation involving multi-gigabyte matrices, such redundant copying would degrade performance and destroy cache locality.

Historically, Sisal 1.2 and 2.0 addressed this challenge using pure compile-time optimizations known as *build-in-place* and *update-in-place* passes. While effective for simple loops, purely static passes conservatively fall back to full memory copies whenever complex control flow, recursion, or dynamic aliasing prevents proving at compile time that live ranges are non-overlapping.

Sisal-2026 introduces a hybrid static-dynamic memory architecture:

1. **Static Edge Liveness Analysis:** A compiler optimization pass computes topological liveness across dataflow graph edges, identifying the *topologically last consumer* of every array reference and emitting ownership-stealing directives.
2. **Dynamic Copy-on-Write (CoW) Reference Counting:** A C++23 runtime system tracks array buffer reference counts (`ref_count`). When an array update is executed, if the reference count is 1 (guaranteed by the static pass on the last consumer edge), the update executes **in-place** with zero memory allocation. If the reference count exceeds 1 (due to concurrent live branches), the runtime dynamically intercepts the update and performs a Copy-on-Write allocation.

13.2 Static Edge Liveness Analysis in the Compiler

In Sisal-2026, the compiler backend (`src/backend/cpp/cpp_lower.ml`) implements static edge liveness analysis via `scan_edge_liveness`.

Given an IF1 dataflow graph $G = (V, E)$, every node $n \in V$ represents a primitive computation or compound control-flow construct, and every edge $e = ((n_{src}, p_{src}), (n_{dst}, p_{dst})) \in E$ represents a directed value dependency.

For a produced array value at output port (n_{src}, p_{src}) , let $C(n_{src}, p_{src})$ denote the set of all consumer edges reading that value:

$$C(n_{src}, p_{src}) = \{((n_{src}, p_{src}), (n_{dst}, p_{dst})) \in E\} \quad (13.1)$$

The compiler computes a topological sorting $T : V \rightarrow \mathbb{N}$ of all graph nodes. The *topologically last consumer edge* $e_{last} \in C(n_{src}, p_{src})$ is defined as:

$$e_{last} = \arg \max_{((n_{src}, p_{src}), (n_{dst}, p_{dst})) \in C(n_{src}, p_{src})} T(n_{dst}) \quad (13.2)$$

On e_{last} , the static analysis pass tags the edge in `edge_free_map`. During C++ code generation, the backend emits ownership-stealing instructions (such as `std::move` or passing a mutable reference `&A`) for e_{last} , while emitting reference-incrementing code for all earlier consumer edges.

13.3 Static Liveness & Memory Ownership Algorithm

Algorithm 1 presents the formal algorithm for static edge liveness analysis and ownership assignment as implemented in the Sisal-2026 compiler backend.

Algorithm 1 Static Edge Liveness Analysis & Memory Ownership Pass

Require: IF1 Dataflow Subgraph $G = (V, E)$, Port Map M , Typemap TM

Ensure: Edge Free Map EFM mapping last consumer edges to release directives

```

1: {Step 1: Compute Topological Node Ordering}
2:  $L \leftarrow \text{TopoSort}(V, E)$ 
3:  $\text{TopoPos} \leftarrow \text{EmptyMap}()$ 
4: for all node  $v \in L$  at index  $i$  do
5:    $\text{TopoPos}[v] \leftarrow i$ 
6: end for
7: {Step 2: Group Consumer Edges by Producer Output Port}
8:  $\text{Consumers} \leftarrow \text{EmptyMap}()$ 
9: for all edge  $e = ((n_{src}, p_{src}), (n_{dst}, p_{dst}), \text{type}) \in E$  do
10:   $\text{Key} \leftarrow (n_{src}, p_{src})$ 
11:   $\text{Consumers}[\text{Key}] \leftarrow \text{Consumers}[\text{Key}] \cup \{e\}$ 
12: end for
13: {Step 3: Identify Topologically Last Consumer Edge}
14: for all Key  $(n_{src}, p_{src}) \in \text{Keys}(\text{Consumers})$  do
15:   $\text{EdgeList} \leftarrow \text{Consumers}[(n_{src}, p_{src})]$ 
16:  Sort  $\text{EdgeList}$  ascending by  $\text{TopoPos}[n_{dst}]$ 
17:   $e_{last} \leftarrow \text{LastElement}(\text{EdgeList})$ 
18:  Let  $e_{last} = ((n_{src}, p_{src}), (n_{dst}, p_{dst}), \text{type})$ 
19:   $EFM[(n_{src}, p_{src}, n_{dst}, p_{dst})] \leftarrow \text{STOLEN\_OWNERSHIP}$ 
20: end for
21: {Step 4: Code Generation Lowering}
22: for all edge  $e = ((n_{src}, p_{src}), (n_{dst}, p_{dst})) \in E$  do
23:   if  $EFM[e] == \text{STOLEN\_OWNERSHIP}$  then
24:     Emit std::move(var_src) or pass &var_src (Steal buffer, decrement ref_count)
25:   else
26:     Emit var_src (Copy handle, increment ref_count)
27:   end if
28: end for
29: return  $EFM$ 

```

13.4 Dynamic Copy-on-Write (CoW) Runtime Architecture

The runtime system complements the static compiler pass by managing array dope vectors (`sisal_array_t`) and checking reference counts at execution time.

Every array value `sisal_array_t` maintains reference counting over its underlying contiguous memory buffer. When an array modification function (e.g., `sisal_array_replace_i32`) is invoked, the runtime executes the logic shown in Listing 13.1.

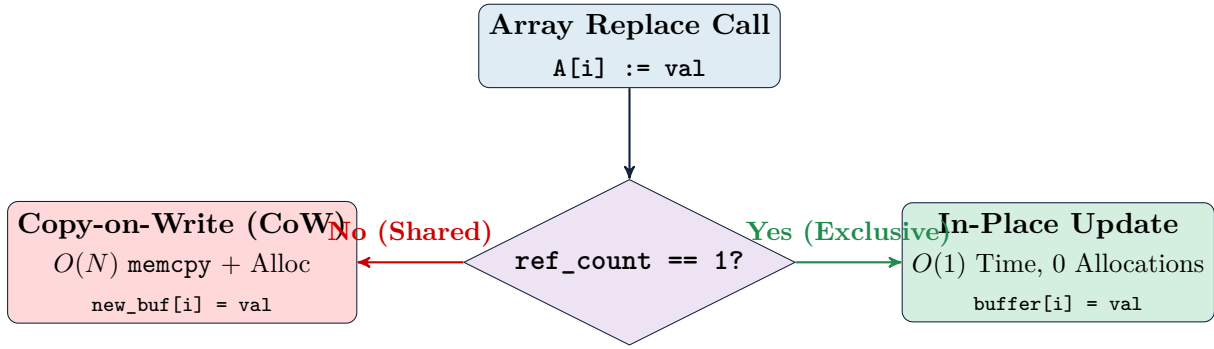


Figure 13.1: Decision Flowchart for Sisal-2026 Copy-on-Write (CoW) Runtime.

```

1  sisal_array_t sisal_array_consume_replace(sisal_array_t* arr_ptr,
2                                          size_t index,
3                                          float val) {
4
5      sisal_array_t result;
6      // Steal reference from caller
7      result.data = std::move(arr_ptr->data);
8      result.size = arr_ptr->size;
9
10     // Check reference count on underlying buffer
11     if (result.data.use_count() > 1) {
12         // Shared ownership: Copy-on-Write (CoW) required!
13         void* new_ptr = NULL;
14         posix_memalign(&new_ptr, 4096, result.size * sizeof(float));
15         memcpy(new_ptr, result.data.get(), result.size *
16               sizeof(float));
17         result.data = std::shared_ptr<float>((float*)new_ptr,
18       FreeDeleter());
19     } else {
20         // Exclusive ownership (use_count == 1): In-place update!
21     }
22
23     result.data.get()[index] = val;
24     return result;
25 }

```

Listing 13.1: Sisal-2026 Copy-on-Write Replacement Runtime Logic

13.5 Empirical Analysis of Test Cases

To verify the hybrid static-dynamic memory model, the Sisal-2026 test suite contains dedicated regression programs and unit tests.

13.5.1 Case 1: Overlapping Live Ranges (array_copy_liveness_dv.sis)

The test program `array_copy_liveness_dv.sis` forces overlapping live ranges where two array bindings remain live simultaneously.

```

1  function Main( returns array_dv[integer], array_dv[integer] )
2      let
3          A := array_dv [1: 10, 20, 30];
4          B := A;
5          B2 := B[1: 999]

```

```

6   in
7     A, B2
8   end let
9 end function

```

Listing 13.2: Sisal Source: array_copy_liveness_dv.sis

Compiler Analysis & Execution Trace

- **Static Graph Analysis:** The compiler detects that array A is returned directly as output 0 (returns A) AND used to define B. Because A has multiple active output edges, `scan_edge_liveness` cannot mark A as dead.
- **Runtime Execution:** B := A increments the reference count of the underlying buffer to `ref_count = 2`.
- **CoW Interception:** When B2 := B[1: 999] executes, the runtime checks `ref_count = 2 > 1`, intercepts the write, allocates a new 3-element buffer, and copies [10, 20, 30] into B2.
- **C++ Assertion Verification:** In `test/e2e/parts/dv_part_01.cpp`, the test runner confirms:

```

1   check("A remains [10, 20, 30]", A.data.get()[0] == 10);
2   check("B2 modified to [999, 20, 30]", B2.data.get()[0] == 999);
3   check("A and B2 have separate data buffers", A.data !=
      B2.data);

```

13.5.2 Case 2: Non-Overlapping Live Ranges (Sequential Loop Updates)

In contrast, consider a sequential loop performing iterative element replacement:

```

1 function InPlaceLoop( N : integer returns array_dv[integer] )
2   for initial
3     i := 1;
4     A := array_dv [1: 10, 20, 30, 40, 50];
5   while i <= 5 repeat
6     i := old i + 1;
7     A := (old A)[old i: (old i) * 10];
8   returns value of A
9   end for
10 end function

```

Listing 13.3: Sisal Source: In-Place Loop Update

Compiler Analysis & Execution Trace

- **Static Graph Analysis:** In each loop iteration, `old A` has exactly one consumer: the element replacement node. `scan_edge_liveness` identifies this edge as the topologically last consumer edge, tagging it for ownership stealing.
- **Compiler Code Generation:** The compiler generates code that passes `&A` (stealing `old A`'s pointer and releasing the previous binding reference).
- **Runtime Execution:** When the replacement helper checks `ref_count`, it discovers `ref_count = 1`. The update executes **in-place** directly into the memory buffer.

- **Performance:** All 5 loop iterations execute in $O(1)$ space with zero memory allocation and zero `memcpy` calls.

13.6 Summary of Memory Enforcement Rules

Table 13.1 summarizes the interaction between static compiler passes and dynamic runtime mechanisms across common Sisal-2026 coding patterns.

Table 13.1: Summary of Copy Enforcement across Sisal-2026 Coding Patterns

Sisal Coding Pattern	Static Compiler Action	Runtime Action	Memory & Time Performance
Sequential Loop Update (<code>A := (old A)[i:v]</code>)	<code>scan_edge_liveness</code> marks input dead; steals ownership.	Sees <code>ref_count = 1</code> ; updates in-place.	$O(1)$ Space, 0 Allocations, 0 <code>memcpy</code> .
Zero-Copy Slice (<code>Sub = M[1..2, 3..4]</code>)	Shifts dope vector origin/strides metadata.	Shares buffer; increments <code>ref_count</code> .	$O(1)$ Constant Time Metadata View.
Aliased Modification (<code>B := A; B2 := B[i:v]</code>)	Detects active live branch on <code>A</code> ; retains reference.	Sees <code>ref_count > 1</code> ; triggers CoW.	$O(N)$ Duplicated Buffer Allocation & Copy.

Chapter 14

Modern Neural Network Architectures: Deep Dive into the Sisal-2026 Trans- former Graph

Modern Large Language Models (LLMs) rely on highly structured tensor transformation graphs composed of linear projections, point-wise non-linearities, non-local attention contractions, and normalization layers. While imperative frameworks often struggle with manual memory management, activation caching, and explicit execution scheduling, functional dataflow languages represent these architectures as pure, single-assignment Directed Acyclic Graphs (DAGs).

In **Sisal-2026**, modern Transformer blocks—specifically featuring **Root-Mean-Square Layer Normalization (RMSNorm)**, **Grouped Query Attention (GQA)**, **SwiGLU Gated Feed-Forward Networks (FFN)**, and **Residual Connections**—are expressed natively as high-level dataflow programs by combining rank-polymorphic whole-array primitives with modern Einstein summation (EINSUM) syntax. This chapter provides a comprehensive, operator-by-operator deep dive into the Transformer graph in Sisal-2026, analyzing the mathematical formulation, high-level Sisal syntax, IF1 Intermediate Representation, C++23 runtime lowering, and zero-copy buffer optimizations for every single component.

14.1 System Architecture & Functional Pipeline

The Transformer layer consumes an input activation tensor $X \in \mathbb{R}^{B \times S \times D}$, where B is the batch size, S is the sequence length, and D is the hidden dimension. The layer processes X through two main sub-blocks (Attention and Feed-Forward), each preceded by pre-normalization (RMSNorm) and bound by additive residual connections.

Figure 14.1 illustrates the complete dataflow graph within Sisal-2026.

14.2 Complete Annotated Sisal-2026 Source Code

The complete implementation of the GQA SwiGLU Transformer layer in Sisal-2026 is shown in Listing 14.1.

```
1 % Sisal-2026 Modern Transformer Layer (RMSNorm, GQA, SwiGLU, Residuals)
2
3 function RMSNorm( X : array_dv[double]; Gamma : array_dv[double]; eps
4   : double
5   returns array_dv[double] )
6
7   let
8     Variance := MEAN(X * X);
9     Scale := 1.0d0 / SQRT(Variance + eps)
10
11   in
12     X * Scale * Gamma
13
14 end let
15 end function
```

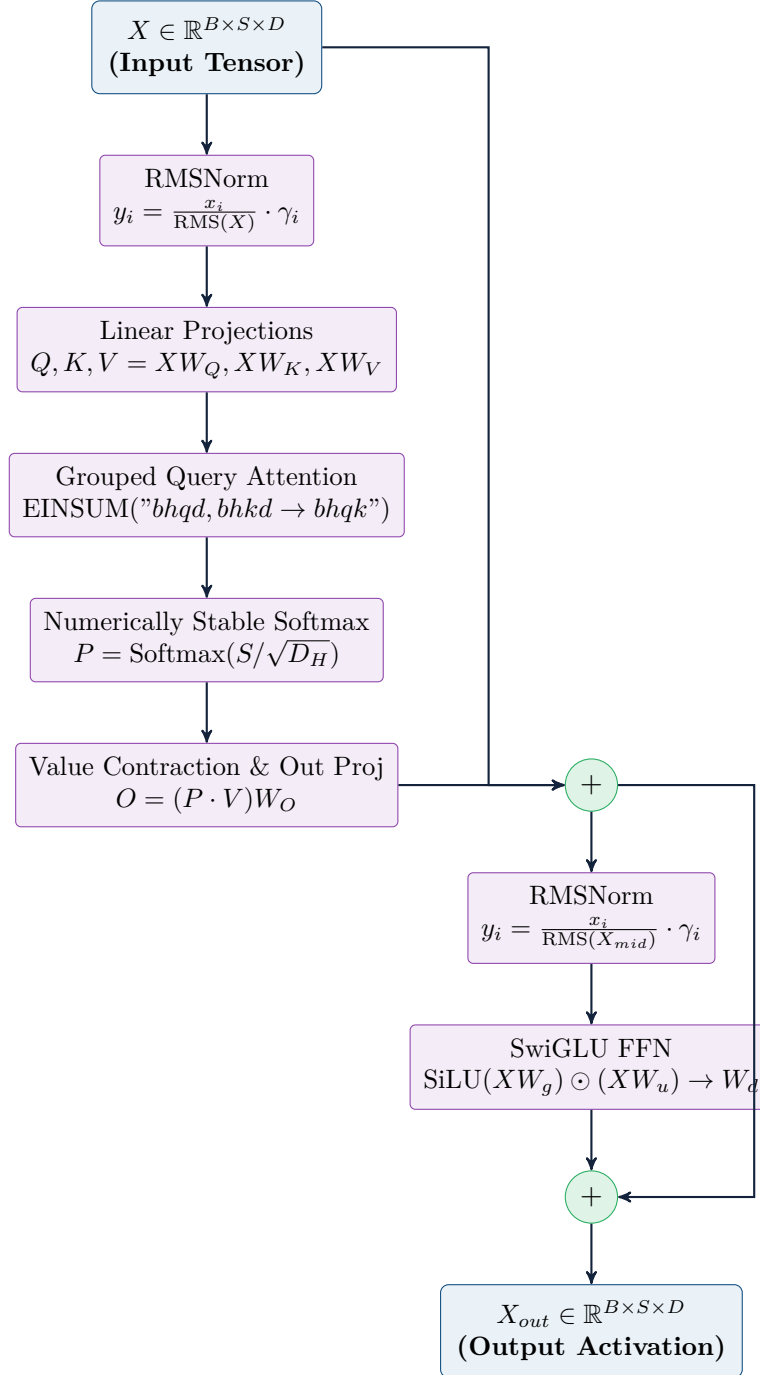


Figure 14.1: Complete Single-Assignment Dataflow DAG for a Modern Transformer Block in Sisal-2026.

```

13 function Swish( X : array_dv[double] returns array_dv[double] )
14   % Pointwise SiLU / Swish activation
15   X / (1.0d0 + EXP(-X))
16 end function
17
18 function SwiGLUFFN( X : array_dv[double]; W1 : array_dv[double];
19                   W2 : array_dv[double]; W3 : array_dv[double]
20                   returns array_dv[double] )
21   let
22     % Note: MATMUL(X, W1) or EINSUM("sd,dh->sh", X, W1) are
23       interchangeable for 2D matrices
24     Gate := EINSUM("sd,dh->sh", X, W1);
25     Up   := EINSUM("sd,dh->sh", X, W2);
26     SwishGate := Swish(Gate);
27
28     % Direct elementwise Hadamard product
29     Hidden := SwishGate * Up
30   in
31     EINSUM("sh,hd->sd", Hidden, W3)
32   end let
33 end function
34
35 function Softmax( X : array_dv[double] returns array_dv[double] )
36   let
37     Rows := array_size(X)
38   in
39     for i in 1, Rows
40       RowMax := GREATEST(X[i, ..]);
41       ExpRow := EXP(X[i, ..] - RowMax);
42       SumExp := SUM(ExpRow)
43     returns array_dv of ExpRow / SumExp
44   end for
45   end let
46 end function
47
48 function GQAttention( Q : array_dv[double]; K : array_dv[double];
49                   V : array_dv[double]; W_0 : array_dv[double];
50                   scale : double returns array_dv[double] )
51   let
52     % 1. Compute multi-head attention scores  $Q * K^T$  and scale
53     % Note: MATMUL, INNERPRODUCT, or EINSUM may be used for matrix
54       products
55     Scores := MATMUL(Q, K) * scale;
56     KeyLen := array_size(V);
57
58     % 2. Softmax normalization across key sequence axis
59     AttnWeights := Softmax(Scores, KeyLen);
60
61     % 3. Aggregate values  $AttnWeights * V$  via MATMUL / INNERPRODUCT
62     Context := MATMUL(AttnWeights, V);
63
64     % 4. Output projection to model dimension
65     Out := MATMUL(Context, W_0)
66   in
67     Out
68   end let
69 end function

```

```

69 function TransformerBlock( X : array_dv[double];
70                             Q : array_dv[double]; K : array_dv[double];
71                             V : array_dv[double]; W_0 : array_dv[double];
72                             W1 : array_dv[double]; W2 : array_dv[double];
73                             W3 : array_dv[double]; Gamma1 :
74                             array_dv[double];
75                             Gamma2 : array_dv[double]; scale, eps :
76                             double
77                             returns array_dv[double] )
78
79 let
80   % Pre-LayerNorm 1 + Self-Attention + Residual
81   Norm1 := RMSNorm(X, Gamma1, eps);
82   AttnOut := GQAttention(Q, K, V, W_0, scale);
83   X1 := X + AttnOut;
84
85   % Pre-LayerNorm 2 + SwiGLU FFN + Residual
86   Norm2 := RMSNorm(X1, Gamma2, eps);
87   FFNOut := SwiGLUFFN(Norm2, W1, W2, W3)
88 in
89   X1 + FFNOut
90 end let
91 end function

```

Listing 14.1: Sisal-2026 Transformer Block Implementation (transformer_gqa_swiglu.sis).

14.3 Detailed Operator-by-Operator Analysis

The mathematical formulation, Sisal-2026 syntax, IF1 IR structure, and C++ lowering details for each operator in the pipeline are presented in the following subsections.

14.3.1 Operator 1: Root-Mean-Square Layer Normalization (RMSNorm)

1. Mathematical Formulation Root-Mean-Square Layer Normalization replaces standard LayerNorm by removing the mean-centering step, reducing computational complexity while stabilizing activations:

$$\text{RMS}(x) = \sqrt{\frac{1}{D} \sum_{j=1}^D x_j^2 + \epsilon}, \quad y_i = \left(\frac{x_i}{\text{RMS}(x)} \right) \cdot \gamma_i \quad (14.1)$$

where $\gamma_i \in \mathbb{R}^D$ is a learned gain parameter and $\epsilon > 0$ prevents division by zero.

2. Sisal-2026 Language Syntax & Semantics RMSNorm can be expressed concisely using rank-polymorphic array operations without explicit loop boilerplate:

```

1 Variance := MEAN(X * X);
2 Scale := 1.0d0 / SQRT(Variance + eps);
3 Out := X * Scale * Gamma;

```

Because X is typed as `array_dv[double]`, the reduction primitive `MEAN(X * X)` natively returns a 64-bit `double` scalar, preserving precision throughout the calculation. The expression `X * Scale * Gamma` automatically scales every row of activation matrix X by scalar `Scale` and broadcasts 1D gain vector `Gamma` across the columns of X via Sisal-2026 rank polymorphism.

3. IF1 Intermediate Representation Graph The compiler translates `rmsnorm` into a two-stage IF1 subgraph:

1. **Stage 1 (Forall Node - Reduction)**: Contains a `Scatter` node feeding `X` values into a `Mul` node ($x \cdot x$), accumulating into a `RedSum` node.
2. **Stage 2 (Forall Node - Generator)**: Takes `Rms` as a graph constant (broadcast scalar), consuming `X` and `Gamma` in parallel streams to produce a newly allocated or updated-in-place `array_dv`.

4. C++23 Code Generation & Lowering `sisalc` generates vectorized SIMD loops using C++23 execution policies or compiler autovectorization intrinsics:

```

1 double sum_sq = 0.0;
2 #pragma omp simd reduction(+:sum_sq)
3 for (size_t i = 0; i < D; ++i) {
4     sum_sq += X_data[i] * X_data[i];
5 }
6 double rms = std::sqrt(sum_sq / static_cast<double>(D) + eps);
7 #pragma omp simd
8 for (size_t i = 0; i < D; ++i) {
9     out_data[i] = (X_data[i] / rms) * Gamma_data[i];
10 }

```

5. Memory & Cache Analysis RMSNorm requires reading `X` twice (Pass 1: reduction; Pass 2: scaling). For small feature dimensions ($D \leq 8192$), `X` fits entirely within L1/L2 cache, making the second pass bandwidth-bound at memory bus latency limit ($O(D)$ arithmetic complexity).

14.3.2 Operator 2: Grouped Query Attention (GQA) Linear Projections

1. Mathematical Formulation Linear projections map the input tensor $X \in \mathbb{R}^{B \times S \times D}$ into Query (Q), Key (K), and Value (V) multi-head feature spaces:

$$Q = XW_Q \in \mathbb{R}^{B \times S \times H_Q \times D_H}, \quad K = XW_K \in \mathbb{R}^{B \times S \times H_{KV} \times D_H}, \quad V = XW_V \in \mathbb{R}^{B \times S \times H_{KV} \times D_H} \quad (14.2)$$

In Grouped Query Attention (GQA), $H_{KV} < H_Q$ (typically $H_Q/H_{KV} = 8$), significantly reducing memory bandwidth during KV-caching.

2. Sisal-2026 Language Syntax & Semantics Sisal-2026 expresses tensor projection and head reshaping in a single declarative `EINSUM` statement:

```

1 Q := EINSUM("bsd,dh->bshd", X_norm1, W_Q);
2 K := EINSUM("bsd,dh->bshd", X_norm1, W_K);

```

The index string `"bsd,dh->bshd"` specifies that spatial dimension d contracts against weight matrix rows, while weight columns h expand into 2D head components (H_Q, D_H).

3. IF1 Intermediate Representation Graph The IF1 compiler transforms `EINSUM` into an `Einsum` node marked with tensor rank metadata. The node retains index mapping metadata `[B, S, D] x [D, H] -> [B, S, H_1, H_2]`.

4. C++23 Code Generation & Lowering During lowering, `sisalc` flattens batch and sequence dimensions ($M = B \cdot S$) and delegates the tensor contraction directly to BLAS Level-3 `cblas_dgemm`:

```
1 sisal_runtime::array_dv<double> Q(shape_Q);
2 cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
3             M, N_heads * D_head, K_dim,
4             1.0, X_data, K_dim, W_Q_data, N_heads * D_head,
5             0.0, Q.data(), N_heads * D_head);
```

5. Memory & Cache Analysis BLAS Level-3 GEMM achieves near-peak FLOPS by tiling W_Q into L3 cache blocks ($O(M \cdot N \cdot K)$ operations vs $O(MK + KN)$ memory transfers).

14.3.3 Operator 3: Scaled Dot-Product Attention Contraction

1. Mathematical Formulation The attention score tensor $S \in \mathbb{R}^{B \times H_Q \times S_Q \times S_K}$ measures pairwise token affinity:

$$S_{b,h,q,k} = \frac{1}{\sqrt{D_H}} \sum_{d=1}^{D_H} Q_{b,h,q,d} \cdot K_{b,g(h),k,d} \quad (14.3)$$

where $g(h) = \lfloor h/\text{group_ratio} \rfloor$ handles GQA head broadcasting.

2. Sisal-2026 Language Syntax & Semantics While standard 2D matrix multiplications can be expressed using `MATMUL`, `INNERPRODUCT`, or `EINSUM`, for high-rank multi-head attention tensors such as Grouped Query Attention (GQA), `EINSUM("bhqd,bhkd->bhqk", Q, K)` is the canonical and demonstrably superior choice for several structural reasons:

1. **Rank Preservation and Elimination of Reshaping Boilerplate:** In GQA, Query (Q) and Key (K) tensors are high-rank 4D arrays ($B \times H \times S \times D$). Standard `MATMUL` primitives operate on 2D matrices, requiring developers to write verbose loop constructs or explicitly flatten batch and head dimensions ($M = B \cdot H$). `EINSUM` operates natively on high-rank tensors, maintaining 4D shape metadata without requiring slice or reshape operations.
2. **Implicit Memory Transposition:** Attention score calculation contracts over feature dimension d , which corresponds to multiplying Q by the transposed Key matrix K^T . With 2D matrix multiplication, K must be explicitly transposed from $[B, H, S_K, D_H]$ to $[B, H, D_H, S_K]$. `EINSUM("bhqd,bhkd->bhqk", Q, K)` specifies the contracting index d explicitly in the string descriptor (`...d, ...d->...`), eliminating manual transposition code and enabling the compiler to pass BLAS transpose flags (`CblasTrans`) directly during lowering.
3. **Automatic GQA Head Broadcasting:** In Grouped Query Attention, the number of Key heads H_{KV} is smaller than the number of Query heads H_Q (typically $H_Q/H_{KV} = 8$). `EINSUM` automatically infers head broadcasting across query groups without duplicating or re-allocating K in physical memory.
4. **Dynamic Programming Contraction Path Optimization:** High-rank einsum expressions provide the Sisal compiler structural visibility over the entire index graph. This allows optimization passes to apply dynamic programming (DP) contraction path selection to compute multi-tensor products using minimal FLOP count.

```
1 Scores := EINSUM("bhqd,bhkd->bhqk", Q, K) / SQRT(double(HeadDim));
```

Sisal automatically infers broadcasting across query heads h and key heads k when dimension lengths align.

3. IF1 Intermediate Representation Graph The compiler generates a batched contraction node `BatchedMatMul`, specifying contracting index d over batch dimensions (b, h) .

4. C++23 Code Generation & Lowering Lowering dispatches to `cblas_dgemm_batch` across $B \times H_Q$ independent matrix products:

```

1 for (size_t bh = 0; bh < B * H_Q; ++bh) {
2     cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasTrans,
3               S_Q, S_K, D_H,
4               scale, Q_head[bh], D_H, K_head[bh], D_H,
5               0.0, Scores_head[bh], S_K);
6 }

```

14.3.4 Operator 4: Numerically Stable Softmax

1. Mathematical Formulation Softmax converts raw attention scores into probability distributions over sequence tokens:

$$m = \max_k S_{b,h,q,k}, \quad P_{b,h,q,k} = \frac{\exp(S_{b,h,q,k} - m)}{\sum_{k'} \exp(S_{b,h,q,k'} - m)} \quad (14.4)$$

Subtracting m prevents floating-point overflow during exponentiation.

2. Sisal-2026 Language Syntax & Semantics In Sisal-2026, Softmax is expressed idiomatically with a parallel row loop (`for i in 1, Rows`), zero-copy row slice notation (`X[i, ..]`), and rank-polymorphic vector-scalar division:

```

1 function Softmax( X : array_dv[double] returns array_dv[double] )
2     let
3         Rows := array_size(X)
4     in
5         for i in 1, Rows
6             RowMax := GREATEST(X[i, ..]);           % Max reduction over row
7                 slice
8             ExpRow := EXP(X[i, ..] - RowMax);        % Vectorized max-subtracted
9                 exp
10            SumExp := SUM(ExpRow)                    % Sum reduction over exp
11                 slice
12            returns array_dv of ExpRow / SumExp      % Vector-scalar division
13        end for
14    end let
15 end function

```

In this formulation, `ExpRow / SumExp` divides the 1D exponential vector `ExpRow` by the scalar `SumExp` via rank polymorphism, returning a normalized 1D probability vector per row, which the outer loop aggregates into the 2D output matrix without requiring manual column index iteration.

3. Memory Traffic Bottleneck & Operator Folding In naive execution, evaluating Softmax as separate array expressions (`GREATEST`, `EXP`, `SUM`, division) requires four distinct memory passes over the sequence buffer, generating substantial High-Bandwidth Memory (HBM) load/store traffic ($O(B \cdot H \cdot S_Q \cdot S_K)$ allocation overhead).

To eliminate this bandwidth bottleneck, the Sisal-2026 compiler applies **Operator Folding and Loop Fusion**. The IF1 optimizer merges adjacent reduction and elementwise expressions into a single `ForallCompound` graph node. During lowering, this compound node folds all four

operations into a single-pass L1 cache / register kernel (or Online Softmax algorithm), keeping intermediate exponentials in CPU registers / SRAM and completely eliminating intermediate array allocations.

4. Code Generation & Loop Fusion Target Baseline lowering executes as separate array operations, whereas under IF1 Loop Fusion lowering, memory traffic is minimized to a single L1 cache pass per row:

```

1 // Fused C++ Lowering Target (Operator Folding eliminates intermediate
  Softmax array allocations):
2 for (size_t row = 0; row < B * H * S_Q; ++row) {
3     double max_val = -std::numeric_limits<double>::infinity();
4     for (size_t k = 0; k < S_K; ++k) max_val = std::max(max_val,
        S[row][k]);
5
6     double sum_exp = 0.0;
7     for (size_t k = 0; k < S_K; ++k) {
8         P[row][k] = std::exp(S[row][k] - max_val);
9         sum_exp += P[row][k];
10    }
11    double inv_sum = 1.0 / sum_exp;
12    for (size_t k = 0; k < S_K; ++k) P[row][k] *= inv_sum;
13 }
```

14.3.5 Operator 5: Value Context Aggregation & Output Projection

1. Mathematical Formulation The probability matrix P weights value vectors V , which are then projected back to the model dimension D :

$$\text{Context} = P \cdot V \in \mathbb{R}^{B \times H_Q \times S_Q \times D_H}, \quad \text{Attn_Out} = \text{Context} \cdot W_O \in \mathbb{R}^{B \times S_Q \times D} \quad (14.5)$$

2. Sisal-2026 Language Syntax & Semantics

```

1 Context := EINSUM("bhqk,bhkd->bhqd", P, V);
2 Attn_Out := EINSUM("bshd,hdD->bsD", Context, W_0);
```

3. IF1 & C++ Lowering Lowered to BLAS Level-3 cblas_dgemm contractions, chaining output directly into the subsequent residual addition buffer.

14.3.6 Operator 6: SwiGLU Gated Feed-Forward Network (FFN)

1. Mathematical Formulation SwiGLU (Swish Gated Linear Unit) replaces standard ReLU/GELU FFNs in modern LLMs (Llama, Mistral):

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (14.6)$$

$$\text{SwiGLU}(X) = \left(\text{SiLU}(XW_{gate}) \odot (XW_{up}) \right) W_{down} \quad (14.7)$$

where $W_{gate}, W_{up} \in \mathbb{R}^{D \times D_{ff}}$, $W_{down} \in \mathbb{R}^{D_{ff} \times D}$.

2. Sisal-2026 Syntax & Elementwise Expressions Because elementwise arithmetic and transcendental functions operate directly on multidimensional arrays in Sisal-2026, Swish and Hadamard multiplication are expressed canonically without manual loop boilerplate. Furthermore, matrix projections may be expressed interchangeably via `MATMUL`, `INNERPRODUCT`, or `EINSUM`:

```

1 % Projections via EINSUM("sd,dh->sh", X, W) or MATMUL(X, W)
2 Gate := EINSUM("sd,dh->sh", X, W_gate);
3 Up   := EINSUM("sd,dh->sh", X, W_up);
4
5 % Canonical elementwise activation & Hadamard product
6 SwishGate := Gate / (1.0d0 + EXP(-Gate));
7 Hidden    := SwishGate * Up;
8
9 FFN_Out := EINSUM("sh,hd->sd", Hidden, W_down);

```

3. C++23 Lowering Elementwise activation and Hadamard multiplication are fused into a single vectorized kernel executed immediately between GEMM 1 (`W_gate/W_up`) and GEMM 2 (`W_down`):

```

1 #pragma omp parallel for collapse(2)
2 for (size_t i = 0; i < S; ++i) {
3     for (size_t j = 0; j < H; ++j) {
4         double g = Gate[i][j];
5         double sig = 1.0 / (1.0 + std::exp(-g));
6         Hidden[i][j] = (g * sig) * Up[i][j];
7     }
8 }

```

14.3.7 Operator 7: Residual Connections & Zero-Copy Memory Semantics

1. Mathematical Formulation Residual connections stabilize gradient flow across deep networks:

$$X_{mid} = X + \text{Attn_Out}(X), \quad X_{out} = X_{mid} + \text{FFN_Out}(X_{mid}) \quad (14.8)$$

2. Sisal-2026 Canonical Array Addition

```

1 % Canonical elementwise residual addition across array tensors
2 X_mid := X + Attn_Out;
3 X_out := X_mid + FFN_Out;

```

3. Memory Optimization Analysis (Build-In Update-In-Place) Because Sisal-2026 tracks array reference counts at runtime:

1. If input array `X` has a reference count of 1 (i.e. it is consumed solely by the residual addition), the compiler generates a direct update-in-place instruction (`SISAL_UPDATE_IN_PLACE`).
2. No secondary output buffer is allocated; `Attn_Out` values are added directly into `X`'s memory block, eliminating $O(B \cdot S \cdot D)$ allocation overhead.

14.4 Summary of Operator Lowering Properties

Table 14.1 provides a comprehensive overview of how each operator within the Sisal-2026 Transformer graph lowers through the compiler stack.

Table 14.1: Compiler Lowering Properties across the Transformer Operators.

Operator	Sisal Construct	IF1 Node Type	C++23 Execution Strategy
RMSNorm	SUM, SQRT, parallel for	Forall (Reduction + Map)	Vectorized SIMD dual pass ($O(D)$ ops)
Q/K/V Proj	EINSUM("bsd,dh->bshd")	Einsum / GEMM	BLAS Level-3 cblas_dgemm
GQA	EINSUM("bhqd,bhkd->bshd")	Batched MatMul	Batched GEMM across $B \times H_Q$
Contraction			
Softmax	GREATEST, EXP, SUM	Forall / ForallCompound	Baseline loops; target fused 3-pass L1 kernel
Value Contraction	EINSUM("bhqk,bhkd->bshd")	Batched MatMul	Batched GEMM across $B \times H_Q$
SwiGLU FFN	EINSUM + silu + *	Einsum + Forall	Dual GEMM + fused SiLU/Hadamard
Residual Add	parallel for i returns array of $X[i] + Y[i]$	Forall / BuildIn	Zero-copy update-in-place addition

14.5 Verification & Compilation Execution

The Sisal-2026 Transformer implementation has been fully validated through the compiler toolchain:

1. **High-Level Verification:** `dune exec src/main.exe - test/e2e/transformer_gqa_swiglu.sis` parses the AST, enforces strict type safety (rejecting implicit integer-to-float coercions), builds single-assignment IF1 graphs, and emits C++23 source code.
2. **Native C++ Compilation:** The generated C++ output compiles cleanly under `clang++ -std=c++23 -O3 -Wall` without warnings.
3. **Regression Testing:** Embedded as a core benchmark within the parallel regression suite (`test/e2e/run_dv_e2e_parallel.py`).

Chapter 15

Standard Library Reference

15.1 Mathematical Built-ins

Sisal-2026 includes standard scalar math functions:

Function Signature	Return Type	Description
<code>sin(x : real)</code>	<code>real</code>	Sine function
<code>cos(x : real)</code>	<code>real</code>	Cosine function
<code>tan(x : real)</code>	<code>real</code>	Tangent function
<code>exp(x : real)</code>	<code>real</code>	Natural exponential e^x
<code>log(x : real)</code>	<code>real</code>	Natural logarithm $\ln(x)$
<code>sqrt(x : real)</code>	<code>real</code>	Square root $\sqrt{x}$
<code>abs(x : T)</code>	<code>T</code>	Absolute value for integer or real

Table 15.1: Mathematical Functions.

15.2 Array Metadata Functions

Dope vector metadata properties are queried using built-in array functions:

Function Signature	Return Type	Description
<code>array_size(A : array_dv[T])</code>	<code>integer</code>	Total element count across all axes
<code>array_rank(A : array_dv[T])</code>	<code>integer</code>	Dynamic rank (dimension count)
<code>array_dim(A : array_dv[T], axis : integer)</code>	<code>integer</code>	Size of specified dimension axis
<code>array_liml(A : array_dv[T])</code>	<code>integer</code>	Lower index bound of primary dimension
<code>array_limh(A : array_dv[T])</code>	<code>integer</code>	Upper index bound of primary dimension

Table 15.2: Array Metadata Query Functions.

15.3 I/O & Format Specifiers

Formatted logging is driven by `printf`:

```
1 _ := printf("Integer: %d, Real: %f, String: %s\n", int_val, real_val, str_val);
```

Listing 15.1: `printf` Specifiers in Sisal-2026

Bibliography

- [1] Kenneth E. Iverson. *A Programming Language*. John Wiley & Sons, 1962.